

DataSpace: Benchmarking Data Agents for Verifiable Analytics over Heterogeneous Workspaces

Boyan Li*
HKUST(GZ)
Guangzhou, China

Zhuowen Liang*
HKUST(GZ)
Guangzhou, China

Yupeng Xie*
HKUST(GZ)
Guangzhou, China

Xiaotian Lin
HKUST(GZ)
Guangzhou, China

Tianqi Luo
HKUST(GZ)
Guangzhou, China

Xinyu Liu
HKUST(GZ)
Guangzhou, China

Yizhang Zhu
HKUST(GZ)
Guangzhou, China

Zhangyang Peng
HKUST(GZ)
Guangzhou, China

Yuan Li
Tsinghua University
Beijing, China

Zhengxuan Zhang
HKUST(GZ)
Guangzhou, China

Jiayi Zhang
HKUST(GZ)
Guangzhou, China

Nan Tang
HKUST(GZ)
Guangzhou, China

Guoliang Li
Tsinghua University
Beijing, China

Yuyu Luo
HKUST(GZ)
Guangzhou, China

Abstract

Data agents enable natural-language analytics over organizational workspaces, where relevant evidence may be scattered across databases, structured files, long documents, and multimedia. Existing benchmarks largely isolate structured querying, retrieval, or open-ended analysis, leaving heterogeneous evidence discovery, complete tabular outputs, and deterministic evaluation insufficiently unified. We introduce DATASPACE, a benchmark in which data agents produce verifiable tabular results from task-local heterogeneous workspaces. It contains 410 cross-language tasks and 7,439 artifacts totaling 15.01 GB across CSV, JSON, SQLite, Markdown, PDF, and video. DATASPACE also served as the official evaluation benchmark for the KDD Cup 2026 Data Agents for Complex Data Analysis competition. Each agent receives only a question and workspace and returns the complete requested tabular result. We construct DATASPACE with DATASPACE-BUILDER, an execution-grounded framework comprising cross-language transformation, constraint-aware relational sampling, modality routing and artifact rendering, and human review and task repair by 11 domain experts. A deterministic evaluator performs header-invariant column alignment, type- and precision-aware normalization, and order-aware row comparison. Across six recently released frontier multimodal models and five widely used agent harnesses, the best accuracy reaches 66.34%, while harness choice creates a 15.36-point spread with the backbone fixed. Multimodal evidence integration and joins consistently reduce accuracy across all six backbones. These results show that DATASPACE remains unsaturated and identify key challenges for improving data-agent reliability.

1 Introduction

Data agents are emerging as a natural-language interface to organizational data [17, 24, 37, 40, 44, 45, 47, 49, 57, 58, 62]. In realistic analytical settings, however, the information needed to answer a user

*These authors contributed equally to this work.

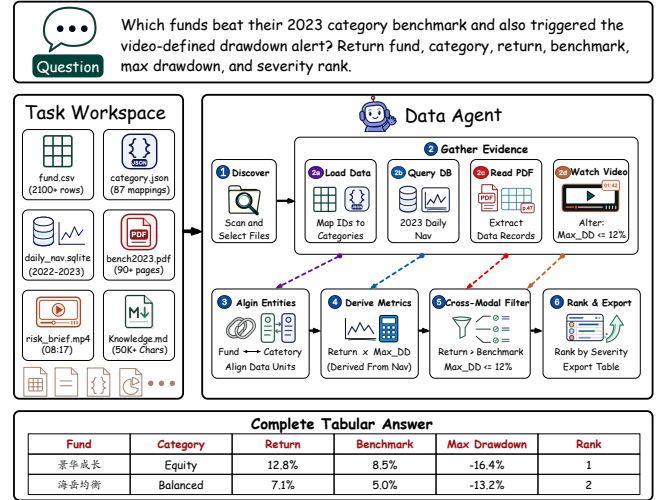


Figure 1: The DATASPACE task interface, illustrated with a fund-risk task. The agent combines an alert rule from video, category benchmarks extracted from a long PDF, and daily NAV queried from SQLite, then aligns entities, computes the requested metrics, and returns the complete tabular result.

question rarely resides in a single clean table or a preselected database [7, 35, 48]. The question and its evidence may cross languages and representations, spanning relational databases, structured and semi-structured files, business documents, and multimedia artifacts alongside valid but irrelevant files. An effective data agent therefore acts as a *workspace solver*: it inspects the available data, selects sources and tools, aligns information across representations, executes multi-step computations, and returns a result that the user can directly consume.

Existing benchmarks capture complementary parts of this setting. *Structured-data benchmarks*, from Spider [55] to Spider 2.0 [13],

Table 1: Comparison of representative benchmarks in artifact coverage, workspace demands, and answer/evaluation semantics.

Type	Benchmark	#Tasks	Input artifacts				Workspace requirements				Output and evaluation			
			DB	Files	Docs	Media	Cross-art.	Discovery	Long docs	Doc→records	Cross-lang.	Complete table	Model-free	Schema-inv.
Structured	WikiTableQuestions [33]	22,033	–	✓	–	–	–	–	–	–	–	–	✓	–
	Spider [55]	10,181	✓	–	–	–	–	–	–	–	–	✓	✓	–
	BIRD [19]	12,751	✓	–	–	–	–	–	–	–	–	✓	✓	–
	Spider 2.0 [13]	632	✓	–	✓	–	✓	✓	–	–	–	✓	✓	–
Unstructured	HotpotQA [53]	113K	–	–	✓	–	✓	✓	–	–	–	–	✓	–
	CRAG [52]	4,409	–	–	✓	–	✓	✓	–	–	–	–	–	–
	MMLongBench-Doc [27]	1,062	–	–	✓	–	–	–	✓	–	–	–	✓	–
Data agent	DABStep [7]	450+	–	✓	✓	–	✓	✓	–	–	–	–	✓	–
	KramaBench [11]	104	–	✓	✓	–	✓	✓	–	–	–	–	–	–
	LongDA [20]	505	–	✓	✓	–	✓	✓	✓	–	–	–	✓	–
	DataCross [35]	200	✓	✓	✓	–	✓	✓	–	✓	–	–	–	–
	FDABench [48]	2,007	✓	✓	✓	✓	✓	✓	–	–	–	–	–	–
Ours	DATASPACE	410	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Notes. Input/workspace checkmarks denote explicit coverage; output/evaluation checkmarks denote benchmark-wide requirements. Files: standalone structured/semi-structured artifacts; Docs: textual/visual documents; Media: audio/video. Cross-art.: multi-artifact/system integration; Discovery: sources not preselected; Long docs: explicit long-document processing; Doc→records: document fields/records feed downstream analysis; Cross-lang.: joint question–workspace input. Complete table: correctness requires the complete result table, submitted directly or obtained by query execution; Model-free: no LLM judge; Schema-inv.: columns align despite header wording or order.

offer strong tests of relational reasoning with deterministic evaluation, but generally identify the relevant table or database in advance. *Unstructured-data benchmarks* such as MMLongBench-Doc [27] introduce long, visually rich inputs, yet focus primarily on evidence retrieval, grounding, and answer synthesis. *Data-agent benchmarks* move closer to open-ended analytical workspaces: KramaBench [11] studies data-to-insight pipelines over data lakes, while FDABench [48] extends analysis across structured data, documents, and media. As summarized in Table 1, these advances have not yet unified three properties central to realistic data analysis: **(L1) Workspace scope**: a task-local workspace spanning structured files, databases, long documents, and multimedia artifacts, with language variation across both the question and data; **(L2) Output contract**: a consistent objective requiring the complete analytical result rather than a factoid, pipeline, or open-ended report; and **(L3) Evaluation semantics**: deterministic evaluation that accepts equivalent representations while rejecting incomplete or erroneous answers.

We therefore introduce DATASPACE¹, a benchmark for verifiable data analytics over self-contained heterogeneous workspaces (Figure 1). An agent receives only a natural-language question and a task-local workspace, autonomously discovers and combines the available data, and returns the complete requested tabular result. DATASPACE contains 410 cross-language tasks and 7,439 artifacts totaling approximately 15 GB. Chinese and English may occur across both the user question and its workspace artifacts. The workspaces span CSV, JSON, SQLite, Markdown, PDF, and Video, and each task is paired with a complete tabular reference answer. **DATASPACE also served as the official evaluation benchmark for the KDD Cup 2026 Data Agents for Complex Data Analysis competition** [10].

To construct such cross-language, heterogeneous workspaces reliably, we propose DATASPACE-BUILDER, an execution-grounded framework that transforms instances from EHRSQL [12] and BULL [56], two English Text-to-SQL benchmarks covering clinical and financial analytics. Their relational databases supply domain

data, while executable SQL provides parseable analytical logic and execution-based validation. DATASPACE-BUILDER comprises four stages: *Cross-Language Transformation*, *Constraint-Aware Relational Sampling*, *Modality Routing & Artifact Rendering*, and *Human Review & Task Repair*. Together, these stages transform source questions, databases, and query logic into cross-language, heterogeneous workspace tasks and derive their reference answers through execution. The resulting question, workspace, reference answer, and evaluation semantics are cross-reviewed by a panel of 11 domain experts, with disputed cases discussed and repaired before release. We also design a deterministic evaluator that aligns columns regardless of header wording or position, normalizes equivalent value formats, and compares rows according to whether their order matters for the task.

Contributions. Our contributions are:

- **A heterogeneous workspace benchmark.** We introduce DATASPACE, comprising 410 cross-language tasks and six modalities under a uniform, exactly verifiable tabular-output objective.
- **An execution-grounded construction framework.** We develop DATASPACE-BUILDER, which transforms executable Text-to-SQL resources into task-local heterogeneous workspaces with expert review and repair.
- **A semantics-aware evaluator.** We provide deterministic evaluation of complete tabular results that tolerates equivalent representations while rejecting incomplete or erroneous outputs.
- **An empirical study of data agents.** We establish baselines across six frontier multimodal models and five agent harnesses; the best reaches 66.34% accuracy, harness choice produces a 15.36-point spread, and multimodal evidence and joins consistently reduce performance.

2 Related Work

Structured-data benchmarks. Natural-language analytics over structured data is studied through table question answering and Text-to-SQL [14–16, 18, 22, 23, 25, 34, 60, 61]. Table QA benchmarks such as WikiTableQuestions [33] predict denotations over semi-structured tables. Spider [55] and BIRD [19] cover cross-domain and large-scale databases, while EHRSQL [12] and BULL [56] capture

¹Code: <https://github.com/HKUSTDial/DataSpace>
Dataset: <https://huggingface.co/datasets/HKUSTDial/DataSpace>

clinical and financial analytics. Spider 2.0 [13] further introduces enterprise artifacts around the SQL workflow. Executable queries enable deterministic result-level evaluation, but these settings provide limited coverage of evidence discovery and reconciliation across separate files, documents, and media.

Unstructured-data benchmarks. HotpotQA [53] and CRAG [52] test retrieval and synthesis over multi-hop or retrieval-augmented corpora, while FinanceBench [9] and MMLongBench-Doc [21, 27] target financial reports and long, visually rich documents. HybridQA [5], MultiModalQA [42], and Video-MME [8] add linked tables, passages, images, or video. These settings provide strong tests of evidence localization, cross-page reasoning, and perception, but generally target factoids, short lists, choices, or free-form responses. They rarely require recovering typed record collections from long documents and combining them with other workspace data to produce a complete tabular result.

Data-agent benchmarks. DABStep [7], KramaBench [11], and LongDA [4, 20] cover multistep processing, data-lake discovery, and long-document navigation. Data Agent Benchmark [26] and AgenticDataBench [41] emphasize multi-system querying or recurring data-science skills, while DataCross [35] and FDABench [6, 48, 59] incorporate visual tables and media. This family is closest to our setting, but its targets range from factoid answers and executable pipelines to choices and reports, accompanied by execution-, rubric-, or model-based evaluation [43, 51]. DATASPACE instead holds the task contract fixed: every task requires a complete tabular result scored by the same deterministic protocol.

3 Benchmark Overview and Task Formulation

DATASPACE evaluates a data agent as a workspace solver: given an analytical question and the contents of a task-local workspace, the agent must return the complete typed table requested by the user. This section defines the task interface, summarizes the benchmark scope, and identifies the capabilities exercised by this setting.

3.1 Task Formulation

For task i , the public input is

$$x_i = (q_i, \mathcal{W}_i), \quad \mathcal{W}_i = \mathcal{W}_i^{\text{str}} \cup \mathcal{W}_i^{\text{doc}} \cup \mathcal{W}_i^{\text{med}}, \quad (1)$$

where q_i is a natural-language question. The workspace contains structured and semi-structured artifacts $\mathcal{W}_i^{\text{str}}$ (CSV, JSON, and SQLite), document artifacts $\mathcal{W}_i^{\text{doc}}$ (Markdown and PDF), and media artifacts $\mathcal{W}_i^{\text{med}}$ (video). The initial observation o_0 gives the agent the question and access to the workspace root; artifact contents are acquired as the agent interacts with the workspace.

Let $\mathcal{T}_A$ denote the tools available to agent A . They may include file-system inspection, structured-data parsing, SQL execution, code execution, document extraction, and video understanding. The action space, agent state, and nonterminal environment transition are jointly defined as

$$\begin{aligned} \mathcal{A}_A &= \{\text{Call}(\tau, \theta) : \tau \in \mathcal{T}_A, \theta \in \Theta_\tau\} \\ &\cup \{\text{Answer}(Y) : Y \text{ is a tabular result}\}, \\ h_t &= (o_0, a_0, \dots, a_{t-1}, o_t), \quad s_t = (h_t, m_t), \\ a_t &= \pi_A(s_t), \quad (o_{t+1}, m_{t+1}) = \text{Exec}_i(s_t, a_t), \end{aligned} \quad (2)$$

where Θ_τ is the argument space of tool τ , m_t is working memory, and π_A is the agent policy. For a tool-call action, Exec_i executes the selected tool in the fixed workspace and returns its observation. The terminal action is $a_T = \text{Answer}(\mathcal{Y}_i)$, where $\mathcal{Y}_i$ is a tabular result serialized as a CSV file. Figure 1 illustrates the resulting task-level input–output interface.

The benchmark-side record additionally contains the reference result $\mathcal{Y}_i$ and a compact evaluation configuration c_i . The benchmark record and binary task score are

$$b_i = (x_i, \mathcal{Y}_i, c_i), \quad s_i = \mathbf{1}[\hat{\mathcal{Y}}_i \equiv_{c_i} \mathcal{Y}_i]. \quad (3)$$

The configuration records the semantic type of each reference column, numeric comparison rules where needed, and whether row order is significant. It belongs to the evaluation protocol rather than the question–workspace input. A task is correct when its complete tabular prediction matches the reference result under these semantics; missing or extra rows and columns make the prediction incorrect even when some returned values match. Section 4.5 specifies the equivalence criterion and the aggregate metric.

3.2 Benchmark Scope

DATASPACE contains 410 tasks spanning financial, macroeconomic, and healthcare analytics. Its workspaces combine CSV, JSON, SQLite, Markdown, PDF, and video in 13 modality combinations, and questions and artifacts may mix Chinese and English. Every sampled source table remains represented: long documents may encode records or fields, while video may supply a condition, intermediate value, or answer. Answer tables range from a single cell to multi-column outputs with thousands of rows.

Tasks couple four capabilities: **(i) workspace discovery**; **(ii) interpretation and alignment** of types, schemas, entities, units, and languages; **(iii) relational computation**, such as filtering, joining, aggregation, ranking, and temporal reasoning; and **(iv) complete tabular-result materialization**.

4 Benchmark Construction and Evaluation

In this section, we introduce DATASPACE-BUILDER, a four-stage pipeline that transforms Text-to-SQL instances into reviewed heterogeneous workspace tasks (Figure 2). Its stages are Cross-Language Transformation, Constraint-Aware Relational Sampling, Modality Routing & Artifact Rendering, and Human Review & Task Repair. We then present the evaluation protocol for finalized tasks. Further implementation details, construction costs, and running examples appear in Appendix A.

4.1 Cross-Language Transformation

Source corpora. We use EHRSQL [12] and BULL [56] as source corpora. These English Text-to-SQL benchmarks cover clinical and financial analytics, respectively. Each selected source instance provides a natural-language question q_0 , a relational database $\mathcal{D}_0$, and executable SQL σ_0 .

Joint transformation. Both source corpora are English-only at the question–database level, whereas our task setting varies language across the complete question–workspace pair. Translating the question or database in isolation can misalign entity names,

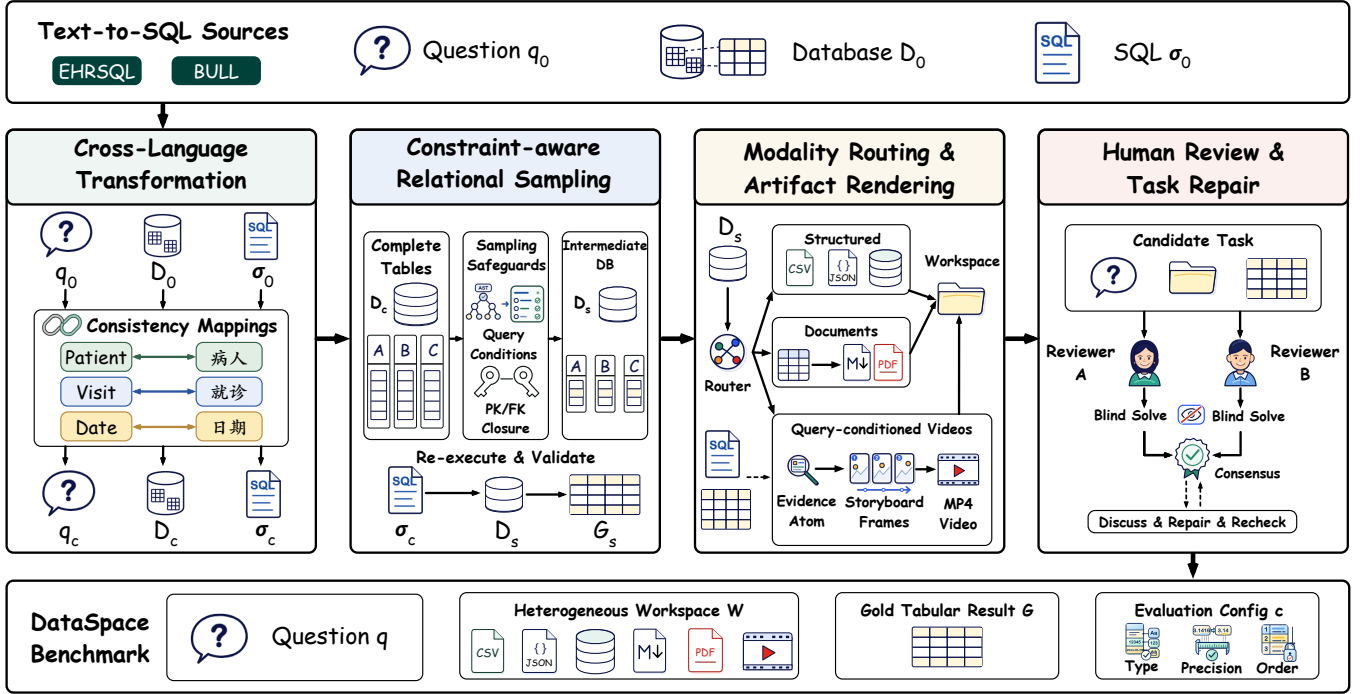


Figure 2: Overview of DATASPACE-BUILDER. Text-to-SQL instances pass through Cross-Language Transformation, Constraint-Aware Relational Sampling, Modality Routing & Artifact Rendering, and Human Review & Task Repair before being frozen as heterogeneous benchmark records.

predicate values, and executable SQL. We therefore treat cross-language transformation (CLT) as a joint migration of the question, database state, and executable workload. For a source tuple (D_0, q_0, σ_0) , we independently choose the target languages ℓ_D and ℓ_q for the database and question:

$$(\mathcal{D}_c, q_c, \sigma_c; M) = \text{CLT}_{\ell_D, \ell_q}(\mathcal{D}_0, q_0, \sigma_0), \quad (4)$$

where $\mathcal{D}_c$, q_c , and σ_c are the transformed database, question, and SQL, respectively, and M is a materialized replacement map.

Consistency-aware rewriting. To keep the same field or entity consistent wherever it appears, we link columns connected by a foreign key, a shared name, or substantial value overlap. Each linked group is translated jointly, while identifiers, codes, URLs, dates, and numbers remain unchanged. LLM-generated translations are stored as table-, column-, and cell-level mappings $M = \{M_{\text{tab}}, M_{\text{col}}, M_{\text{val}}\}$. We use M to deterministically rewrite the database and SQL and translate the question with the same terminology.

Validation. We retain a transformed tuple only after structural and execution checks. The rewritten SQL must reference existing translated entities and execute on $\mathcal{D}_c$, and its result must satisfy $\text{Exec}(\mathcal{D}_c, \sigma_c) \cong \tau_M(\text{Exec}(\mathcal{D}_0, \sigma_0))$, where τ_M applies the induced value translation to the source execution result. Execution equivalence verifies database-SQL consistency but cannot establish that q_c preserves the original intent; we therefore use an LLM judge to verify question-SQL semantic alignment. Instances that fail either check are repaired and re-executed or rejected before relational data sampling.

4.2 Constraint-Aware Relational Sampling

Task-local data diversification. Source Text-to-SQL datasets commonly associate many questions with a small number of shared database states. Directly reusing these states would produce workspaces with highly repetitive entities, values, and relational neighborhoods. We instead construct task-local relational instances by sampling rows while retaining the complete table inventory and schema. The sampled scale also makes downstream artifact rendering tractable, particularly when complete tables are transformed into long Markdown or PDF documents.

Sampling safeguards. Naively sampling each table independently can remove a condition value, break a join path, or disconnect related entities. We therefore construct a lightweight safeguard set C_s that combines primary keys, foreign keys, and known inter-table relationships with join columns, predicate and boundary values, and target entities extracted from the SQL AST.

Relationally consistent sampling. Given a seeded sampling policy ψ , we retain rows required by query safeguards and then propagate their key values across schema relationships. Remaining rows are selected according to the table-level sampling budget, including rows from tables not referenced by the source SQL.

Materialization and re-execution. All sampled tables are materialized as a task-local intermediate database:

$$\mathcal{D}_s = \text{Sample}(\mathcal{D}_c; \psi, C_s), \quad \mathcal{Y}_s = \text{Exec}(\mathcal{D}_s, \sigma_c). \quad (5)$$

The query result $\mathcal{Y}_s$ becomes the candidate reference. It is not required to equal the source result; sampling may change entities, aggregates, rankings, or result cardinality. A sample is accepted

only if the SQL executes successfully, required relationships remain valid, and the result has not become unintentionally empty or semantically degenerate. Otherwise, the database is resampled, repaired, or rejected.

4.3 Modality Routing & Artifact Rendering

Query-independent base routing. The intermediate database fixes the relational content of a task; this stage changes how agents encounter that content. We first apply a seeded rule-based policy π_r , with fixed seed z_r , that assigns each sampled table to one or more compatible renderers and materializes the base workspace $\mathcal{W}_{\text{base}}$ from $\mathcal{D}_s$. The policy considers schema properties, renderer compatibility, and batch-level modality coverage using only sampled-table metadata. Its base renderers are CSV, JSON, SQLite, Markdown, and PDF. Video is introduced separately as a task-level, query-conditioned augmentation because its construction may depend on the question, executable SQL, and candidate answer. Every sampled table receives a base representation, while the number and sizes of the resulting artifacts follow from the source table inventory, sampled contents, and applicable renderers.

Structured artifact rendering. CSV and record-oriented JSON expose individual sampled tables, whereas a task-local SQLite artifact can retain several related tables and their schema. The renderers preserve headers, cell values, nulls, and duplicate rows; the SQLite renderer additionally preserves declared column types and key relationships. Each output is parsed back into a canonical relation and compared with the rows assigned to that renderer.

Fact-grounded data documents. For a routed table R , let $X_R = \mathcal{D}_s[R]$ denote its complete sampled contents. The LLM-produced plan $p_R = \text{PlanDoc}(X_R)$ specifies a document style, record-identifying columns, semantic attribute groups, and bounded row batches. Document generation then follows

$$\begin{aligned} \mathcal{B}_R &= \text{BuildBlocks}(X_R; p_R), \\ d_R &= \text{Assemble}(\text{DocGen}(\mathcal{B}_R; p_R)). \end{aligned} \quad (6)$$

Each block repeats the identifying columns and supplies an LLM with its exact field names, types, and values. The generated sections are assembled as Markdown d_R and converted to PDF when required. Figure 14 visualizes the alignment from source cells to generation blocks and document spans. The complete running example is provided in Appendix A.3.3.

Query-conditioned video rendering. Following DataMagic [50], we render tabular data as data-insight videos, while conditioning content selection and task integration on the query. For selected tasks, the renderer derives typed evidence atoms from the executable SQL σ_c and sampled tabular result $\mathcal{Y}_s$, then constructs the video and integrates it into the task:

$$\begin{aligned} \mathcal{E}_v &= \text{Select}(\text{AST}(\sigma_c), \mathcal{Y}_s), \\ S_v &= \text{Storyboard}(\mathcal{E}_v), \quad v = \text{Render}(S_v, \text{TTS}(S_v)), \\ (q_r, \mathcal{W}) &= \text{Integrate}(q_c, \mathcal{W}_{\text{base}}, v, \mathcal{E}_v). \end{aligned} \quad (7)$$

The evidence set $\mathcal{E}_v$ contains filter conditions and result cells. A filter condition records a column, operator, and value; a result cell records a row, field, and value. For a multi-step query, *predicate abstraction* assigns a stable condition to a video scene and removes it from the explicit question. For a simple query with a compact

result, *answer-evidence rendering* distributes result cells or an intermediate value across multiple scenes. We revise the question only when evidence is moved to video. Figure 16 shows representative frames from both strategies. Appendix A.3.5 further traces the question, evidence atoms, storyboard scenes, rendered video, and resulting workspace for complete tasks. For tasks without video augmentation, $(q_r, \mathcal{W}) = (q_c, \mathcal{W}_{\text{base}})$.

4.4 Human Review & Task Repair

Execution-based checks verify data–SQL consistency, but cannot determine whether a task is unambiguous or its reference result is correct. We therefore subject every candidate to blind, independent review by two reviewers from a panel of 11 domain experts. Each reviewer first solves the task using only its final question and workspace. After the gold is revealed, both reviewers verify it and independently author the evaluation configuration c_i , covering column types, numeric precision, and ordering. Acceptance requires agreement on the gold and identical canonical configurations. Any disagreement triggers evidence-based discussion and minimal repair of the question, workspace, gold, or configuration, followed by independent rechecking by the same pair. This cycle continues until consensus; unresolved tasks are removed. Appendix A.5 gives the full protocol, and Figure 17 shows the review interface.

4.5 Evaluation Protocol

Task-specific semantics. After human review, each task freezes a configuration $c_i = (o_i, \{v_{ij}\}_{j=1}^{d_i})$, where d_i is the number of reference columns, o_i indicates whether row order is required by the question, and v_{ij} is the canonicalization rule for reference column j . The rule records a semantic type—text, number, date, datetime, or Boolean—and, for numeric columns, the required integer, decimal-place, or significant-digit precision and any percentage convention. It canonicalizes both reference and predicted values: text is normalized to Unicode NFC and trimmed; numbers are parsed as decimals under the configured precision and unit; and dates, datetimes, Booleans, and nulls are converted to canonical values. Appendix A.6 gives the complete protocol, and Figure 18 shows a frozen task configuration.

Header-invariant joint column alignment. Prediction headers are not scored, and predicted columns need not follow the reference order. Instead, the evaluator searches for a one-to-one mapping from reference columns to predicted columns. Let Π_{d_i} be the set of permutations of d_i columns, and let $\pi(j)$ denote the predicted column mapped to reference column j . We use $\widehat{\mathcal{Y}}_i[:, \pi]$ to denote the prediction with its columns reordered by π , and $\text{Canon}_{c_i}(Y)$ to normalize column j of Y with v_{ij} . A mapping is invalid if any predicted cell cannot be interpreted under its target rule.

Answer equivalence. Tabular results with different shapes are unequal. Otherwise, $\text{Rows}_o(Y)$ returns the ordered row sequence when $o = 1$, and the unordered row multiset when $o = 0$; the latter preserves duplicate multiplicities. Define the canonical row view as $\mathcal{V}_i(Y) = \text{Rows}_{o_i}(\text{Canon}_{c_i}(Y))$. The equivalence criterion introduced in Equation 3 is then

$$\widehat{\mathcal{Y}}_i \equiv_{c_i} \mathcal{Y}_i \iff \exists \pi \in \Pi_{d_i} : \mathcal{V}_i(\widehat{\mathcal{Y}}_i[:, \pi]) = \mathcal{V}_i(\mathcal{Y}_i). \quad (8)$$

Column alignment is evaluated over the full result, so a valid mapping must preserve the association among values within every row.

Table 2: Workspace scale of DATASPACE (410 tasks).

Statistic	Overall	Median	P90 / range
Artifacts	7,439 files	20/task	P90: 23; 5–26
Storage	15.01 GB	31.26 MB/task	P90: 74.18 MB
PDF pages	1,088 PDFs; 25,384 pp.	22 pages/PDF	P90: 36
PDF text	28.48M characters	23,232/PDF	P90: 39,522
Markdown	875 files; 26.88M chars	26,190/file	P90: 49,238
Video	189 videos; 5.49 h	103.8 s/video	39.1–158.3 s

Tasks requesting a ranking or another semantically ordered output are compared as row sequences; all other tasks are compared as unordered row multisets.

Aggregate metric. For N tasks, the official metric is Task Accuracy,

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N 1[\hat{\mathcal{Y}}_i \equiv_{c_i} \mathcal{Y}_i]. \quad (9)$$

5 Benchmark Statistics and Analysis

5.1 Composition and Scale

DATASPACE contains 410 tasks: 363 (88.5%) originate from BULL and 47 (11.5%) from EHRSQL. The benchmark covers fund (158 tasks; 38.5%), stock (120; 29.3%), macroeconomic (85; 20.7%), and healthcare analytics (47; 11.5%). Of these tasks, 265 (64.6%) are cross-language and 145 (35.4%) are single-language. Their workspaces span 25,384 PDF pages, 55.36 million PDF and Markdown characters, and 5.49 hours of video. Reference answers contain 126,409 rows; the largest has 12,962 rows, the widest has six columns, and 92 tasks require row order to be preserved. Table 2 summarizes the resulting scale.

5.2 Evidence Requirements

We annotate one minimal, independently sufficient solution path per task to distinguish available artifacts from required evidence. We group CSV/JSON, SQLite, Markdown/PDF, and video as structured-file, database, document, and video carriers.

Available versus required evidence. Figure 3a shows that CSV occurs in every workspace, while JSON, SQLite, Markdown, and PDF each occur in more than 93% of tasks, yet the verified paths use CSV in only 58 tasks and SQLite in 177. Long documents provide required evidence in 135 tasks; among 189 video-enabled workspaces, 97 require video.

Cross-artifact composition. The verified paths use one artifact modality for 276 tasks (67.3%), two for 115 (28.0%), and three for 19 (4.6%); thus, 134 tasks (32.7%) combine multiple modalities. At the carrier-family level in Figure 3b, 129 tasks (31.5%) span multiple carrier families, and 202 (49.3%) require a document or video.

5.3 Analytical Operations

Operation coverage. As shown in Figure 4, projection and filtering are required by 338 (82.4%) and 321 tasks (78.3%), respectively. The benchmark also exercises ordering in 209 tasks (51.0%), aggregation in 146 (35.6%), and joins in 113 (27.6%). Cross-artifact grounding includes document and video understanding in 135 and 97 tasks, and schema and entity alignment in 97 and 90.

Compositional breadth. A task combines between two and eleven nontrivial operation types, with a median of five and a 90th percentile of seven; 248 tasks (60.5%) combine at least five types.

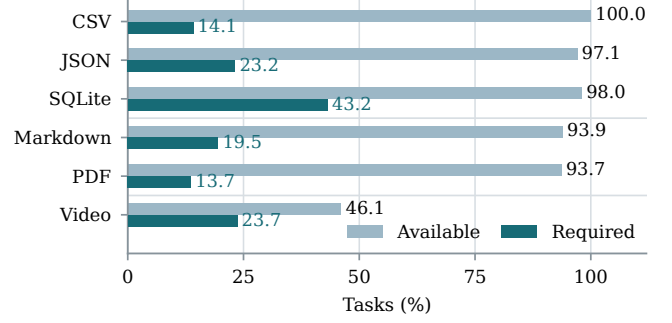
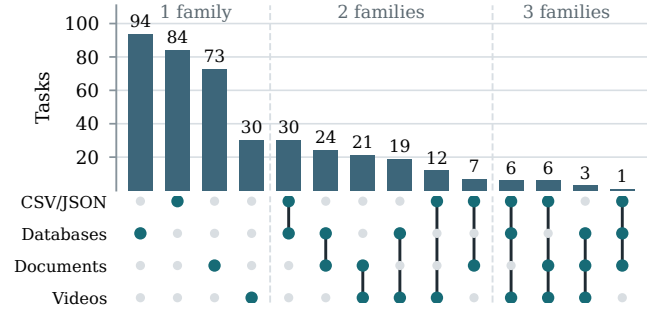
**(a) Available and required modalities.****(b) Required carrier-family intersections.**

Figure 3: Workspace availability and annotated solution evidence. Available denotes presence in the workspace, whereas required denotes use by the verified solution path.

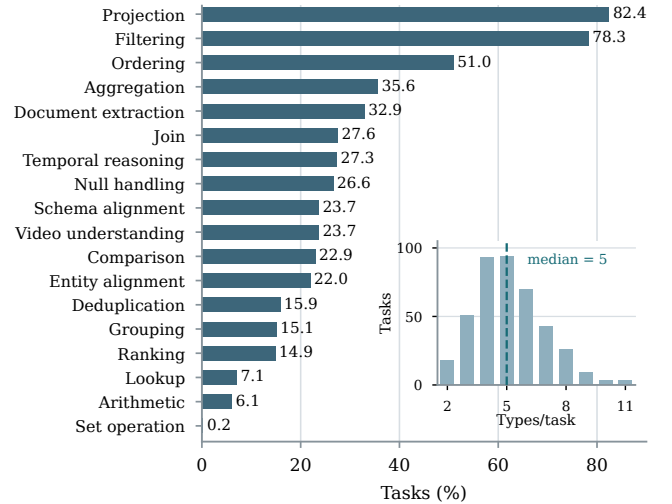


Figure 4: Analytical-operation coverage and compositional breadth. Bars show task-level prevalence; the inset shows distinct operation types per task.

6 Experiments

6.1 Experimental Setup

We evaluate all 410 tasks under two complementary controls. For the *backbone comparison*, we implement DATASPACE-AGENT, a lightweight agent that follows the ReAct [54] paradigm and exposes a minimal set of task-agnostic tools. We fix this agent and vary

Table 3: Overall effectiveness in controlled comparisons. Correct is out of 410; snapshot denotes backbone release month or harness version. Best in each block is bold.

Method	Snapshot	Correct	Acc. (%)
<i>Backbones (DATASPACE-AGENT fixed)</i>			
Grok 4.5 [39]	2026-07	272	66.34
GPT-5.6 Sol [32]	2026-07	265	64.63
Kimi K3 [29]	2026-07	219	53.41
MiMo-V2.5 [1]	2026-04	161	39.27
Claude Sonnet 5 [3]	2026-06	135	32.93
MiniMax M3 [28]	2026-06	117	28.54
<i>Harnesses (MiMo-V2.5 [1] fixed)</i>			
Grok Build [38]	v0.2.106	190	46.34
Claude Code [2]	v2.1.217	183	44.63
DATASPACE-AGENT (<i>ours</i>)	–	161	39.27
Codex [31]	v0.145.0	143	34.88
Smolagents [36]	v1.26.0	127	30.98

only the multimodal backbone, testing six models released between April and July 2026: Grok 4.5 [39], GPT-5.6 Sol [32], Kimi K3 [29], MiMo-V2.5 [1], Claude Sonnet 5 [3], and MiniMax M3 [28]. In the *harness comparison*, we fix MiMo-V2.5 and compare DATASPACE-AGENT, Smolagents [36], Codex [31], Claude Code [2], and Grok Build [38] while retaining each harness’s native planning, tool-use, and context management. Each backbone–harness pair instantiates a data agent. We omit specialized systems that cannot be evaluated faithfully under this protocol, such as DeepAnalyze [58], whose native interface does not cover video inputs or our tabular-output contract, and AgenticData [40], whose original implementation is unavailable. Model and harness configurations, together with the broader compatibility assessment, are provided in Appendix A.7.

Execution protocol. Each run receives the task question and its complete workspace. In the backbone comparison, every task is limited to 60 model turns, 50 tool actions, and 1,800 seconds, with 4 CPUs and 16 GiB of memory. The harness comparison uses the same 1,800-second deadline but does not impose a shared action limit that would override a harness’s native control loop.

Inference and scoring. All backbone calls use the provider-default reasoning configuration through Vercel AI Gateway [46], with a maximum of 32,768 output tokens per call. The official evaluator computes Task Accuracy using the protocol in Section 4.5. Missing predictions, invalid outputs, runtime failures, and exhausted budgets count as incorrect. We additionally record token usage, API cost, tool actions, and wall-clock latency as efficiency diagnostics.

6.2 Overall Effectiveness

Table 3 reports Task Accuracy for both controlled comparisons.

Backbone comparison. With DATASPACE-AGENT fixed, Grok 4.5 achieves the highest observed accuracy of 66.34%, followed by GPT-5.6 Sol at 64.63%; their totals differ by only seven correct tasks. Kimi K3 reaches 53.41%, while the remaining three backbones remain below 40%. The 37.80-point range between the strongest and weakest backbone shows substantial separation, yet even the best result solves only about two thirds of the benchmark. Across the six models, 56 tasks are solved by all models, whereas 76 are solved by none; their oracle union solves 334 tasks (81.46%). Thus,

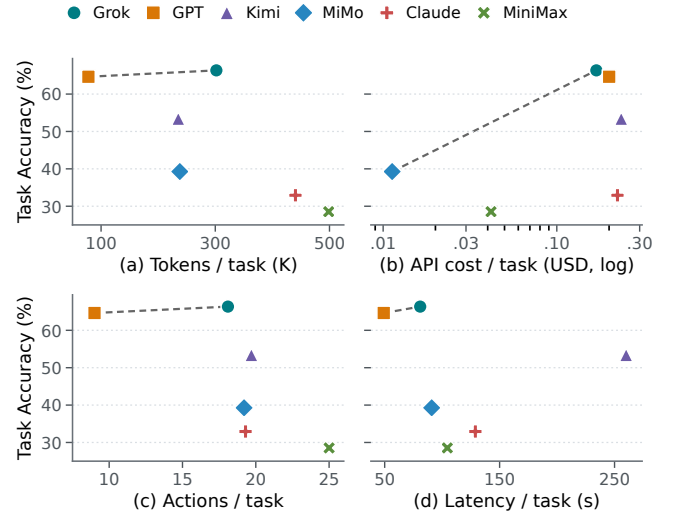


Figure 5: Backbone accuracy–efficiency trade-offs with DATASPACE-AGENT fixed. Dashed lines connect Pareto-efficient points (higher accuracy, lower resource use).

the benchmark contains both a shared hard core and model-specific successes that are hidden by aggregate ranking alone.

Harness comparison. Fixing MiMo-V2.5, Grok Build obtains 46.34% and Claude Code 44.63%, compared with 39.27% for DATASPACE-AGENT, 34.88% for Codex, and 30.98% for Smolagents. The resulting 15.36-point spread demonstrates that the harness substantially affects end-to-end task completion even when the backbone is held constant.

Finding 1. DATASPACE remains unsaturated: the best controlled backbone reaches 66.34% Task Accuracy, while 76 tasks are missed by all six backbones. Harness design also substantially affects agent performance: with MiMo-V2.5 fixed, accuracy ranges from 30.98% to 46.34%.

6.3 Efficiency and Trade-offs

We fix DATASPACE-AGENT to compare backbone efficiency under a common agent design. Figure 5 relates Task Accuracy to mean token usage, API cost, tool actions, and wall-clock latency. GPT-5.6 Sol reaches 64.63% accuracy, only 1.71 points below Grok 4.5, while using 74.2% fewer tokens, 50.3% fewer actions, and 39.2% less wall-clock time per task. These two models form the token-, action-, and latency-based Pareto frontiers: GPT provides the most compact near-top solution, while Grok trades additional resources for the highest accuracy. The cost frontier differs: MiMo-V2.5 costs only \$0.011 per task at 39.27% accuracy, whereas Grok reaches 66.34% at \$0.169 and GPT costs \$0.200 per task. Complete statistics, including medians and tail latency, appear in Appendix A.8.1.

Finding 2. Near-top accuracy need not require long trajectories: GPT-5.6 Sol trails Grok 4.5 by 1.71 points while using 74.2% fewer tokens, 50.3% fewer actions, and 39.2% less latency. Monetary efficiency follows a different frontier, with MiMo-V2.5 providing the lowest-cost operating point at \$0.011 per task.

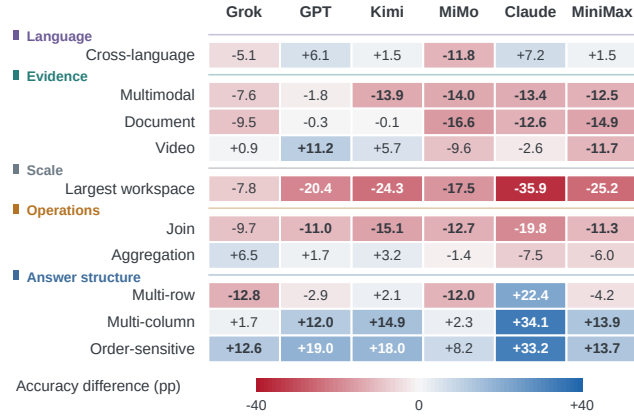


Figure 6: Task Accuracy differences with DATASPACE-AGENT fixed. Cells report percentage-point changes from each row’s reference group; positive values favor the named row.

6.4 Performance across Task Characteristics

We stratify the six DATASPACE-AGENT runs by the annotations in Section 5. Figure 6 reports Task Accuracy gaps across overlapping task characteristics; these are descriptive rather than causal. Sample sizes and raw accuracies appear in Appendix A.8.2.

Language and workspace scale. Cross-language performance varies by backbone: relative to single-language tasks, MiMo-V2.5 declines by 11.8 points and Grok 4.5 by 5.1, whereas GPT-5.6 Sol and Claude Sonnet 5 improve by 6.1 and 7.2 points. The largest workspace quartile underperforms the smallest for all six models, but accuracy across the four quartiles is not monotonic. At the task level, workspace bytes have only a weak negative Spearman correlation (-0.186) with the number of models that solve a task.

Evidence composition. Multimodal tasks underperform single-modal tasks for every backbone by 1.8–14.0 points. Modality presence alone has a less uniform effect: required document evidence is nearly neutral for GPT and Kimi but substantially reduces accuracy for Grok, MiMo, Claude, and MiniMax. Required video evidence helps GPT and Kimi but hurts MiMo, Claude, and MiniMax. These contrasts locate the consistent challenge in cross-modality integration rather than in any single modality.

Relational and answer requirements. Join requirements reduce accuracy for all backbones by 9.7–19.8 points, whereas aggregation has mixed effects. Answer shape is similarly non-monotonic: multi-row answers hurt Grok and MiMo but improve Claude, while multi-column and order-sensitive groups are not less accurate overall. Answer size and order therefore show no uniform negative association with accuracy; the analytical path and task composition remain more informative.

Finding 3. Cross-modality integration and joins are the most consistent sources of degradation: multimodal tasks underperform single-modal tasks for every backbone by 1.8–14.0 points, and joins reduce accuracy by 9.7–19.8 points. Language configuration and answer shape instead exhibit backbone-dependent effects.

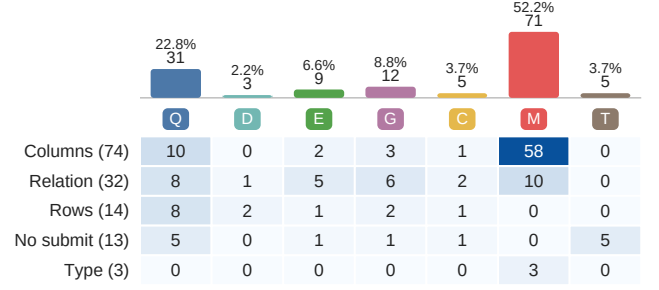


Figure 7: Error analysis of 136 Grok 4.5 failures. Bars show primary causes; the matrix decomposes evaluator symptoms. Q/D/E/G/C/M/T denote intent, discovery, extraction, grounding, computation, materialization, and termination.

6.5 Failure Analysis

We conduct a trace-level error analysis of 136 failures from Grok 4.5, the strongest controlled backbone.

Where failures originate. Answer materialization is the largest category, accounting for 71/136 (52.2%) failures, followed by task specification and intent with 31/136 (22.8%). At the subtype level, 60 materialization failures add or omit columns after the needed internal result is available, while 17 intent failures misformulate the requested output or row grain. These two distinct routes to an incorrect answer schema comprise 77/136 (56.6%) of audited failures. By comparison, only three failures arise from selecting the wrong evidence source; extraction and semantic grounding together account for 21, showing that locating an artifact does not ensure that its values are recovered and aligned correctly.

Symptoms are not diagnoses. Figure 7 shows that the same evaluator outcome can arise at different stages. Of 74 audited column-count mismatches, 58 originate in materialization, while the remainder begin in task intent, extraction, grounding, or computation. Conversely, only 5 of the 13 no-submission outcomes are pure execution-control failures; the other eight follow an earlier persistent interpretation, extraction, grounding, or computation error. Aggregate output symptoms therefore obscure the upstream intervention required to correct a trajectory. The complete subtype taxonomy and counts appear in Appendix A.8.3.

Finding 4. Harnesses must faithfully materialize the exact requested output, rather than merely find and correctly compute the relevant values: target-result misunderstanding and faulty column projection account for 56.6% of Grok 4.5’s audited failures, while only 5 of 13 no-submission outcomes are pure termination failures.

7 Conclusion

We introduced DATASPACE, a 410-task benchmark for verifiable analytics with complete tabular outputs over cross-language, heterogeneous workspaces. DATASPACE-BUILDER transforms executable Text-to-SQL instances into multimodal tasks with expert review, paired with semantics-aware tabular-result evaluation. Across six backbones and five harnesses, the best accuracy reaches 66.34%, while multimodal evidence integration and joins remain key bottlenecks. DATASPACE establishes a rigorous test bed for advancing reliable data agents over heterogeneous workspaces.

References

- [1] 2026. MiMo-V2.5. <https://huggingface.co/collections/XiaomiMiMo/mimo-v25>.
- [2] Anthropic. 2026. How Claude Code Works. Accessed July 27, 2026. <https://code.claude.com/docs/en/how-claude-code-works>
- [3] Anthropic. 2026. Introducing Claude Sonnet 5. <https://www.anthropic.com/news/claude-sonnet-5>
- [4] Yutong Bian, Xianhao Lin, Yupeng Xie, Tianyang Liu, Mingchen Zhuge, Siyuan Lu, Haoming Tang, Jinlin Wang, Jiayi Zhang, Jiaqi Chen, et al. 2025. You Don't Know Until You Click: Automated GUI Testing for Production-Ready Software Evaluation. *arXiv preprint arXiv:2508.14104* (2025).
- [5] Wenhui Chen, Hanwen Zha, Zhiyu Chen, Wenhan Xiong, Hong Wang, and William Yang Wang. 2020. HybridQA: A Dataset of Multi-Hop Question Answering over Tabular and Textual Data. In *EMNLP (Findings)* (Findings of ACL, Vol. EMNLP 2020). Association for Computational Linguistics, 1026–1036.
- [6] Yiyu Chen, Yifan Wu, Shuyu Shen, Yupeng Xie, Leixian Shen, Hui Xiong, and Yuyu Luo. 2025. ChartMark: A Structured Grammar for Chart Annotation. In *2025 IEEE Visualization and Visual Analytics (VIS)*. IEEE, 311–315.
- [7] Alex Egg, Martin Iglesias Goyanes, Friso Kingma, Andreu Mora, Leandro von Werra, and Thomas Wolf. 2025. Dabstep: Data agent benchmark for multi-step reasoning. *arXiv preprint arXiv:2506.23719* (2025).
- [8] Chaoyou Fu, Yuhang Dai, Yongdong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, Peixian Chen, Yanwei Li, Shaohui Lin, Sirui Zhao, Ke Li, Tong Xu, Xiaowu Zheng, Enhong Chen, Caifeng Shan, Ran He, and Xing Sun. 2025. Video-MME: The First-Ever Comprehensive Evaluation Benchmark of Multi-modal LLMs in Video Analysis. In *CVPR*. Computer Vision Foundation / IEEE, 24108–24118.
- [9] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. 2023. FinanceBench: A New Benchmark for Financial Question Answering. *CoRR* abs/2311.11944 (2023).
- [10] KDD Cup 2026. 2026. KDD Cup 2026: Data Agents for Complex Data Analysis. Accessed July 27, 2026. <https://dataagent.top/>
- [11] Eugenie Lai, Gerardo Vitagliano, Ziyu Zhang, Sivaprasad Sudhir, Om Chabra, Anna Zeng, Anton A. Zabreyko, Chenning Li, Ferdi Kossmann, Jialin Ding, Jun Chen, Markos Markakis, Matthew Russo, Weiyang Wang, Ziniu Wu, Michael J. Cafarella, Lei Cao, Samuel Madden, and Tim Kraska. 2025. KramaBench: A Benchmark for AI Systems on Data-to-Insight Pipelines over Data Lakes. *CoRR* abs/2506.06541 (2025).
- [12] Gyubok Lee, Hyeonji Hwang, Seongsu Bae, Yeonsu Kwon, Woncheol Shin, Seongjun Yang, Minjoon Seo, Jong-Yeup Kim, and Edward Choi. 2023. EHRSQL: A Practical Text-to-SQL Benchmark for Electronic Health Records. *CoRR* abs/2301.07695 (2023).
- [13] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In *ICLR*. OpenReview.net.
- [14] Boyan Li, Chong Chen, Zhujun Xue, Yinan Mei, and Yuyu Luo. 2025. DeepEye-SQL: A Software-Engineering-Inspired Text-to-SQL Framework. *CoRR* abs/2510.17586 (2025).
- [15] Boyan Li, Ou Ocean Kun Hei, Yue Yu, and Yuyu Luo. 2026. DPC: Training-Free Text-to-SQL Candidate Selection via Dual-Paradigm Consistency. In *ACL (1)*. Association for Computational Linguistics, 6897–6913.
- [16] Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. 2024. The Dawn of Natural Language to SQL: Are We Fully Ready? [Experiment, Analysis & Benchmark]. *Proc. VLDB Endow.* 17, 11 (2024), 3318–3331.
- [17] Boyan Li, Yiran Peng, Yupeng Xie, Sirong Lu, Yizhang Zhu, Xing Mu, Xinyu Liu, and Yuyu Luo. 2026. DeepEye: A Steerable Self-driving Data Agent System. In *Companion of the International Conference on Management of Data (India) (SIGMOD Companion '26)*. Association for Computing Machinery, New York, NY, USA, 74–77. doi:10.1145/3788853.3801612
- [18] Boyan Li, Jiayi Zhang, Ju Fan, Yanwei Xu, Chong Chen, Nan Tang, and Yuyu Luo. 2025. Alpha-SQL: Zero-Shot Text-to-SQL using Monte Carlo Tree Search. In *ICML (Proceedings of Machine Learning Research, Vol. 267)*. PMLR / OpenReview.net.
- [19] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chen-Chuan Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. In *NeurIPS*.
- [20] Yiyang Li, Zheyuan Zhang, Tianyi Ma, Zehong Wang, Keerthiram Murugesan, Chuxu Zhang, and Yanfang Ye. 2026. LongDA: Benchmarking LLM Agents for Long-Document Data Analysis. *arXiv preprint arXiv:2601.02598* (2026).
- [21] Zhuowen Liang, Xiaotian Lin, Zhengxuan Zhang, Yuyu Luo, Haixun Wang, and Nan Tang. 2026. Long-Document QA with Chain-of-Structured-Thought and Fine-Tuned SLMs. *CoRR* abs/2603.29232 (2026).
- [22] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2025. A Survey of Text-to-SQL in the Era of LLMs: Where Are We, and Where Are We Going? *IEEE Trans. Knowl. Data Eng.* 37, 10 (2025), 5735–5754.
- [23] Xinyu Liu, Shuyu Shen, Boyan Li, Nan Tang, and Yuyu Luo. 2025. NL2SQL-BUGs: A Benchmark for Detecting Semantic Errors in NL2SQL Translation. In *KDD (2)*. ACM, 5662–5673.
- [24] Tianqi Luo, Chuhan Huang, Leixian Shen, Boyan Li, Shuyu Shen, Wei Zeng, Nan Tang, and Yuyu Luo. 2025. nvBench 2.0: Resolving Ambiguity in Text-to-Visualization through Stepwise Reasoning. In *NeurIPS*.
- [25] Peixian Ma, Boyan Li, Runzhi Jiang, Ju Fan, Nan Tang, and Yuyu Luo. 2024. A Plug-and-Play Natural Language Rewriter for Natural Language to SQL. *CoRR* abs/2412.17068 (2024).
- [26] Ruiying Ma, Shreya Shankar, Ruiqi Chen, Yiming Lin, Sepanta Zeighami, Rajoshi Ghosh, Abhinav Gupta, Anushrut Gupta, Tanmai Gopal, and Aditya G Parameswaran. 2026. Can ai agents answer your data questions? a benchmark for data agents. *arXiv preprint arXiv:2603.20576* (2026).
- [27] Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, Pan Zhang, Liangming Pan, Yu-Gang Jiang, Jiaqi Wang, Yixin Cao, and Aixin Sun. 2024. MMLONGBENCH-DQC: Benchmarking Long-context Document Understanding with Visualizations. In *NeurIPS*.
- [28] MiniMax. 2026. MiniMax M3: Frontier Coding, 1M Context, Native Multimodality—All in One Model. <https://www.minimax.io/blog/minimax-m3>
- [29] Moonshot AI. 2026. Kimi K3: Open Frontier Intelligence. <https://www.kimi.com/blog/kimi-k3>
- [30] Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Serkan Ö. Arik, and Tomas Pfister. 2025. MLE-STAR: Machine Learning Engineering Agent via Search and Targeted Refinement. *CoRR* abs/2506.15692 (2025).
- [31] OpenAI. 2026. Codex CLI. Accessed July 27, 2026. <https://learn.chatgpt.com/docs/codex/cli>
- [32] OpenAI. 2026. GPT-5.6 Sol. Accessed July 27, 2026. <https://developers.openai.com/api/docs/models/gpt-5.6-sol>
- [33] Panupong Pasupat and Percy Liang. 2015. Compositional Semantic Parsing on Semi-Structured Tables. In *ACL (1)*. The Association for Computer Linguistics, 1470–1480.
- [34] Wenqi Pei, Shizheng Hou, Boyan Li, Chen Han, Zhichao Shi, and Yuyu Luo. 2026. ROSE: An Intent-Centered Evaluation Metric for NL2SQL. In *ACL (1)*. Association for Computational Linguistics, 5682–5709.
- [35] Ruyi Qi, Zhou Liu, and Wentao Zhang. 2026. DataCross: A Unified Benchmark and Agent Framework for Cross-Modal Heterogeneous Data Analysis. *CoRR* abs/2601.21403 (2026).
- [36] Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. 2025. 'smolagents': a smol library to build great agentic systems. <https://github.com/huggingface/smolagents>.
- [37] Zhihao Shuai, Boyan Li, Siyu Yan, Yuyu Luo, and Weikai Yang. 2026. DeepVIS: Bridging Natural Language and Data Visualization Through Step-Wise Reasoning. *IEEE Trans. Vis. Comput. Graph.* 32, 1 (2026), 868–878.
- [38] SpaceXAI. 2026. Grok Build. Accessed July 27, 2026. <https://docs.x.ai/build/overview>
- [39] SpaceXAI. 2026. Introducing Grok 4.5. <https://x.ai/news/grok-4-5>
- [40] Ji Sun, Guoliang Li, Peiyao Zhou, Yihui Ma, Jingzhe Xu, and Yuan Li. 2025. AgenticData: An Agentic Data Analytics System for Heterogeneous Data. *CoRR* abs/2508.05002 (2025).
- [41] Zhaoyan Sun, Shan Zhong, Daizhou Wen, Jiaxing Han, Guoliang Li, Ying Yan, Peng Zhang, Yu Su, Xiang Qi, Baolin Sun, et al. 2026. AgenticDataBench: A Comprehensive Benchmark for Data Agents. *arXiv preprint arXiv:2607.01647* (2026).
- [42] Alon Talmor, Ori Yoran, Amnon Catav, Dan Lahav, Yizhong Wang, Akari Asai, Gabriel Ilharco, Hannaneh Hajishirzi, and Jonathan Berant. 2021. MultiModalQA: complex question answering over text, tables and images. In *ICLR*. OpenReview.net.
- [43] Yinghao Tang, Xueding Liu, Boyuan Zhang, Tingfeng Lan, Yupeng Xie, Jiale Lao, Yiyao Wang, Haoxuan Li, Tingting Gao, Bo Pan, et al. 2026. IGenBench: Benchmarking the Reliability of Text-to-Infographic Generation. *arXiv preprint arXiv:2601.04498* (2026).
- [44] Yinghao Tang, Yupeng Xie, Yingchaojie Feng, Tingfeng Lan, Jiale Lao, Yue Cheng, and Wei Chen. 2026. ViviDoc: Generating Interactive Documents through Human-Agent Collaboration. *arXiv preprint arXiv:2603.27991* (2026).
- [45] Teable. 2026. Teable AI: Overview. Accessed July 27, 2026. <https://help.teable.ai/en/basic/ai/overview>
- [46] Vercel. 2026. AI Gateway. Accessed July 27, 2026. <https://vercel.com/docs/ai-gateway>
- [47] Jiayi Wang and Guoliang Li. 2025. AOP: Automated and Interactive LLM Pipeline Orchestration for Answering Complex Queries. In *CIDR*. www.cidrdb.org.
- [48] Ziting Wang, Shize Zhang, Haitao Yuan, Jinwei Zhu, Shifu Li, Wei Dong, and Gao Cong. 2025. FDBench: A Benchmark for Data Agents on Analytical Queries over Heterogeneous Data. *CoRR* abs/2509.02473 (2025).
- [49] Yupeng Xie, Yuyu Luo, Guoliang Li, and Nan Tang. 2024. Haichart: Human and AI paired visualization system. *arXiv preprint arXiv:2406.11033* (2024).
- [50] Yupeng Xie, Chen Ma, Zhenyang Wang, Liangwei Wang, Jiayi Zhu, Chuxuan Zeng, Zhouan Shen, Boyan Li, and Yuyu Luo. 2026. DataMagic: Transforming

- Tabular Data into Data Insight Video. *CoRR* abs/2606.20388 (2026).
- [51] Yupeng Xie, Zhiyang Zhang, Yifan Wu, Sirong Lu, Jiayi Zhang, Zhaoyang Yu, Jinlin Wang, Sirui Hong, Bang Liu, Chenglin Wu, et al. 2025. Visjudge-bench: Aesthetics and quality assessment of visualizations. *arXiv preprint arXiv:2510.22373* (2025).
- [52] Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, Lingkun Kong, Brian Moran, Jiaqi Wang, Yifan Ethan Xu, An Yan, Chenyu Yang, Eting Yuan, Hanwen Zha, Nan Tang, Lei Chen, Nicolas Scheffer, Yue Liu, Nirav Shah, Rakesh Wanga, Anuj Kumar, Wen-tau Yih, and Xin Luna Dong. 2024. CRAG - Comprehensive RAG Benchmark. *CoRR* abs/2406.04744 (2024).
- [53] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *EMNLP*. Association for Computational Linguistics, 2369–2380.
- [54] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*. OpenReview.net.
- [55] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir R. Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *EMNLP*. Association for Computational Linguistics, 3911–3921.
- [56] Chao Zhang, Yuren Mao, Yijiang Fan, Yu Mi, Yunjun Gao, Lu Chen, Dongfang Lou, and Jinshu Lin. 2024. FinSQL: Model-Agnostic LLMs-based Text-to-SQL Framework for Financial Analysis. In *SIGMOD Conference Companion*. ACM, 93–105.
- [57] Chao Zhang, Shaolei Zhang, Quehuan Liu, Sabei Chen, Tong Li, and Ju Fan. 2025. TAIJI: MCP-based Multi-Modal Data Analytics on Data Lakes. *CoRR* abs/2505.11270 (2025).
- [58] Shaolei Zhang, Ju Fan, Meihao Fan, Guoliang Li, and Xiaoyong Du. 2025. DeepAnalyse: Agentic Large Language Models for Autonomous Data Science. *arXiv:2510.16872 [cs.AI]* <https://arxiv.org/abs/2510.16872>
- [59] Zhengxuan Zhang, Zhuowen Liang, Yin Wu, Teng Lin, Yuyu Luo, and Nan Tang. 2025. DataMosaic: Explainable and Verifiable Multi-Modal Data Analytics through Extract-Reason-Verify. *CoRR* abs/2504.10036 (2025).
- [60] Yizhang Zhu, Runzhi Jiang, Boyan Li, Nan Tang, and Yuyu Luo. 2025. ElieSQL: Cost-Efficient Text-to-SQL with Complexity-Aware Routing. *CoRR* abs/2503.22402 (2025).
- [61] Yizhang Zhu, Zhangyang Peng, Boyan Li, and Yuyu Luo. 2026. SQLConductor: Search-to-Policy Learning for Step-wise Text-to-SQL Orchestration. *CoRR* abs/2606.23537 (2026).
- [62] Yizhang Zhu, Liangwei Wang, Chenyu Yang, Xiaotian Lin, Boyan Li, Wei Zhou, Xinyu Liu, Zhangyang Peng, Tianqi Luo, Yu Li, Chengliang Chai, Chong Chen, Shimin Di, Ju Fan, Ji Sun, Nan Tang, Fugee Tsung, Jiannan Wang, Chenglin Wu, Yanwei Xu, Shaolei Zhang, Yong Zhang, Xuanhe Zhou, Guoliang Li, and Yuyu Luo. 2025. A Survey of Data Agents: Emerging Paradigm or Overstated Hype? *CoRR* abs/2510.23587 (2025).

A Additional Benchmark Details

A.1 Cross-Language Transformation Details

This appendix expands the cross-language transformation in Section 4.1. We describe the construction contract, algorithmic steps, intermediate artifacts, and normalized prompt templates. Provider-specific system wrappers and batching delimiters are omitted because they do not change the semantic contract. All generative calls use deterministic decoding, and their outputs must pass the stated schema and consistency checks before they can modify a database or workload.

A.1.1 Transformation Contract and Invariants. The input is an executable Text-to-SQL tuple $T_0 = (\mathcal{D}_0, q_0, \sigma_0)$, together with independently selected database and question languages (ℓ_D, ℓ_q) . The output is $T_c = (\mathcal{D}_c, q_c, \sigma_c)$ plus a materialized replacement map M . The transformation enforces four invariants:

- (1) **Referential consistency.** Repeated representations of the same entity receive the same translation across primary keys, foreign keys, denormalized columns, and undeclared join paths.

- (2) **Workload consistency.** Every schema identifier and value literal used by σ_c resolves against $\mathcal{D}_c$.
- (3) **Execution alignment.** Executing σ_c on $\mathcal{D}_c$ reproduces the translated counterpart of the source result, including arity and row multiplicity.
- (4) **Question fidelity.** The transformed question preserves the requested entities, predicates, aggregation, ordering, cardinality, units, and temporal scope of q_0 .

The map M , rather than an unconstrained model response, is the interface between language generation and physical rewriting. Consequently, model calls can be rerun or manually repaired without changing the deterministic migration logic.

A.1.2 Translation-Unit Extraction. Let each text-bearing column be a vertex. We first add an explicit edge between two columns when they participate in a declared foreign-key relationship or share the same normalized name. Connected components of these edges form the initial clusters C_{exp} . To recover undeclared joins, we collect the distinct non-null values $V(C)$ of every cluster and compute the overlap coefficient

$$\rho(C_x, C_y) = \frac{|V(C_x) \cap V(C_y)|}{\min(|V(C_x)|, |V(C_y)|)}. \quad (10)$$

Clusters are merged when $\rho(C_x, C_y) > \theta_{\text{ov}}$ and the intersection contains at least k_{min} distinct values. The second condition prevents a single common token from connecting otherwise unrelated columns. Both parameters are exposed in the construction configuration.

We then remove values that should not be localized. The filters cover nulls, pure numbers, dates and timestamps, URLs, e-mail addresses, file paths, machine-generated identifiers, standardized codes, and values already written in the target language. Domain terms that must remain invariant can be added to a protected glossary. Remaining values are deduplicated within a cluster and divided into bounded prompt chunks. Every item retains its cluster ID and all (table, column) occurrences, so a single translated value can later be fanned out consistently.

Extraction algorithm. The algorithm below separates clustering from prompt-size management; changing the chunk budget B therefore does not change entity equivalence classes.

A.1.3 Replacement-Map Generation. The materialized map contains three scoped mappings:

$$M = \{M_{\text{tab}}, M_{\text{col}}, M_{\text{val}}\}, \quad (11)$$

where table keys are database-scoped, column keys are table-scoped, and value keys retain their column-cluster provenance. Figure 8 shows a normalized serialized form.

All responses are parsed as JSON and checked before map assembly. Each input ID must occur exactly once, protected items must be unchanged, target table names must be unique within a database, and target column names must be unique within a table. A value shared by a cluster receives one target form, which is copied to every listed occurrence. Invalid or incomplete responses are returned to the model together with validator errors under a bounded retry policy; a remaining conflict is repaired manually or the affected

Algorithm 1 Translation-unit extraction**Require:** Schema S , database D , target language ℓ_D **Require:** Thresholds θ_{ov} , k_{min} **Require:** Prompt budget B , protected glossary G **Ensure:** Clusters C , schema units U_s , value chunks U_v

```

1:  $V_T \leftarrow \text{TEXTCOLUMNS}(S)$ 
2:  $C \leftarrow \text{SINGLETONS}(V_T)$ 
3:  $\text{UNIONFKENDPOINTS}(C, S)$ 
4:  $\text{UNIONSAMENAMECOLUMNS}(C, S)$ 
5: repeat
6:    $changed \leftarrow \text{false}$ 
7:   for all unordered  $(C_x, C_y) \in C$  do
8:      $I \leftarrow V(C_x) \cap V(C_y)$ 
9:      $\rho \leftarrow |I| / \min(|V(C_x)|, |V(C_y)|)$ 
10:    if  $\rho > \theta_{ov} \wedge |I| \geq k_{min}$  then
11:       $\text{UNION}(C_x, C_y)$ ;  $changed \leftarrow \text{true}$ 
12:    end if
13:  end for
14: until  $changed = \text{false}$ 
15:  $U_s \leftarrow \text{EXTRACTSCHEMAUNITS}(S, G)$ 
16: for all  $C_j \in C$  do
17:    $V_j \leftarrow \text{DISTINCTNONNULLVALUES}(D, C_j)$ 
18:    $V_j \leftarrow \text{FILTERPROTECTED}(V_j, \ell_D, G)$ 
19:    $V_j \leftarrow \text{DEDUPWITHPROVENANCE}(V_j)$ 
20:    $U_v \leftarrow U_v \cup \text{CHUNK}(V_j, B)$ 
21: end for
22: return  $C, U_s, U_v$ 

```

```

{
  "tables": [
    {
      "source": "...",
      "target": "..."
    }
  ],
  "columns": [
    {
      "table_source": "...",
      "source": "...",
      "target": "..."
    }
  ],
  "values": [
    {
      "cluster_id": "...",
      "source": "...",
      "target": "...",
      "occurrences": [
        {
          "table_source": "...",
          "column_source": "..."
        }
      ]
    }
  ]
}

```

Figure 8: Normalized serialized form of the replacement map.

sample is rejected. Figures 9 and 10 specify the normalized contracts for the two mapping stages.

A.1.4 Deterministic Database Migration. The source database is copied before modification, and all changes are applied inside a transaction. Rewrite order is important because map keys are expressed in source-language identifiers. Cell values are updated first; columns are renamed while source table names still exist; tables are renamed last. Dependent views, indexes, triggers, foreign-key declarations, and schema metadata are then rewritten against the target names. CHECK constraints that contain translated enumerated values are temporarily removed and reconstructed with their mapped literals. Any collision, unresolved reference, or constraint failure aborts the transaction.

```

SYSTEM
You localize relational schemas from <SOURCE_LANGUAGE> to
<TARGET_LANGUAGE>. Produce a faithful terminology map, not a new schema.

RULES
1. Translate only human-readable table and column names.
2. Preserve IDs, standardized codes, SQL keywords, and protected terms.
3. Preserve meaning, domain terminology, granularity, and abbreviations.
4. Table targets must be unique in the database. Column targets must be
   unique within their table.
5. Return JSON only. Do not add, remove, merge, or split input items.

INPUT
DOMAIN: <DATABASE_DOMAIN>
PROTECTED_GLOSSARY: <PROTECTED_GLOSSARY>
SCHEMA_ITEMS:
<LIST_OF_ITEMS_WITH_STABLE_IDS_TABLE_CONTEXT_AND_DESCRIPTIONS>

OUTPUT SCHEMA
{
  "items": [
    {
      "id": "<INPUT_ID>",
      "target": "<TRANSLATION>"
    }
  ]
}

```

Figure 9: Prompt for schema mapping.

```

SYSTEM
Translate database values from <SOURCE_LANGUAGE> to <TARGET_LANGUAGE>
while preserving equality and join semantics.

RULES
1. One source value has exactly one target value within this cluster.
2. Use the same target for every listed table/column occurrence.
3. Do not translate identifiers, codes, URLs, dates, numbers, or entries
   marked PROTECTED.
4. Preserve units, signs, precision, entity identity, and domain meaning.
5. Return JSON only and include every input ID exactly once.

CONTEXT
DOMAIN: <DATABASE_DOMAIN>
CLUSTER_ID: <CLUSTER_ID>
COLUMNS: <TABLE_COLUMN_OCCURRENCES>
SCHEMA_GLOSSARY: <RELEVANT_SCHEMA_MAP>
VALUES: <VALUES_WITH_STABLE_IDS_AND_PROTECTION_FLAGS>

OUTPUT SCHEMA
{
  "cluster_id": "<CLUSTER_ID>",
  "items": [
    {
      "id": "<INPUT_ID>",
      "target": "<TRANSLATION_OR_ORIGINAL>"
    }
  ]
}

```

Figure 10: Prompt for clustered-value mapping.

Updates are scoped by the source table and column recorded in M ; global string replacement is never applied to database contents. When a source value is a substring of another value, exact matching is used for cells and longest-first matching is used only inside parsed schema definitions.

A.1.5 Protect–Replace–Restore SQL Rewriting. SQL rewriting is a workload-migration operation over σ_0 , not a new query-generation call. We parse nested query blocks and build a scope table for base tables, CTEs, aliases, and projected columns. String literals, CTE names, alias declarations, and table-name positions are replaced with typed markers. Schema identifiers are then mapped according to their resolved scope. A string literal is translated only when its comparison context resolves to a column cluster containing the corresponding value entry. This prevents a surface form that appears in two unrelated columns from receiving the wrong replacement. Subqueries are rewritten from the innermost scope outward, after which markers are restored and the resulting SQL is parsed again.

Algorithm 2 Deterministic database migration**Require:** Source database $\mathcal{D}_0$, validated map M **Ensure:** Translated database $\mathcal{D}_c$, synchronized metadata S_c

```

1:  $\mathcal{D}_c \leftarrow \text{TRANSACTIONALCOPY}(\mathcal{D}_0)$ 
2:  $K \leftarrow \text{EXTRACTDEFINITIONS}(\mathcal{D}_c)$ 
3:  $\text{DEFERAFFECTEDCHECKS}(\mathcal{D}_c, K, M)$ 
4: for all  $(t, c, v \mapsto v') \in M_{\text{val}}$  do
5:    $\text{UPDATEEXACTVALUE}(\mathcal{D}_c, t, c, v, v')$ 
6: end for
7: for all  $(t, c \mapsto c') \in M_{\text{col}}$  do
8:    $\text{RENAMECOLUMN}(\mathcal{D}_c, t, c, c')$ 
9: end for
10: for all  $(t \mapsto t') \in M_{\text{tab}}$  do
11:    $\text{RENAME}(\mathcal{D}_c, t, t')$ 
12: end for
13:  $K_c \leftarrow \text{REWRITEDEFINITIONS}(K, M)$ 
14:  $\text{STOREDEFINITIONS}(\mathcal{D}_c, K_c)$ 
15:  $S_c \leftarrow \text{SYNCMETADATA}(\mathcal{D}_c, M)$ 
16:  $\text{ASSERTNO}(\mathcal{D}_c, S_c, M)$ 
17:  $\text{COMMIT}(\mathcal{D}_c)$ 
18: return  $\mathcal{D}_c, S_c$ 

```

Algorithm 3 Protect–replace–restore SQL rewriting**Require:** SQL σ_0 , replacement map M **Require:** Source schema S_0 , target schema S_c **Ensure:** Rewritten SQL σ_c

```

1:  $(ast, scopes) \leftarrow \text{PARSERESOLVE}(\sigma_0, S_0)$ 
2:  $(protected, markers) \leftarrow \text{PROTECT}(ast)$ 
3:  $ordered \leftarrow \text{INNERMOSTTOOUTERMOST}(scopes)$ 
4: for all  $scope \in ordered$  do
5:    $\text{REPLACETABLES}(scope, M_{\text{tab}})$ 
6:    $\text{REPLACECOLUMNS}(scope, M_{\text{col}})$ 
7:   for all literal  $L$  with resolved context  $(t, c)$  do
8:     if  $(t, c, L) \in M_{\text{val}}$  then
9:        $markers[L] \leftarrow M_{\text{val}}[(t, c, L)]$ 
10:    end if
11:   end for
12: end for
13:  $\sigma_c \leftarrow \text{RESTORESERIALIZE}(protected, markers)$ 
14:  $\text{ASSERTPARSEABLE}(\sigma_c)$ 
15:  $\text{ASSERTRESOLVED}(\sigma_c, S_c)$ 
16: return  $\sigma_c$ 

```

The protection layer also prevents accidental substitutions inside SQL keywords, function names, numeric constants, and partial identifiers. When an unqualified column is ambiguous under the current scope, the rewrite is not guessed; it is marked for repair.

A.1.6 Question Translation and Language Composition. Question language is varied independently of database language. The question translator receives the original question, a focused glossary containing only entities relevant to that sample, and optional SQL alignment context. The SQL is construction-time context: it constrains preservation of operators and conditions but is not copied into the natural-language output. No database or SQL artifact is modified during this step. Verified question variants are joined to verified database/SQL variants by stable sample ID, which allows (ℓ_q, ℓ_D) combinations to be assembled without repeating database migration. The normalized translation contract is given in Figure 11.

```

SYSTEM
Translate an analytical question from <SOURCE_LANGUAGE> to
<TARGET_LANGUAGE>. Preserve the exact answer semantics.

PRESERVE
- entities, value literals, units, signs, and numerical thresholds;
- filters, negation, conjunction, and comparison direction;
- aggregation, grouping, distinctness, ranking, ordering, and limits;
- time windows, inclusivity of boundaries, and requested output fields.

RULES
1. Use the supplied target-language glossary for schema and value terms.
2. Write a natural user question; do not mention SQL, schemas, or this task.
3. Do not add explanations, assumptions, or answer values.
4. Return JSON only.

INPUT
SOURCE_QUESTION: <QUESTION>
FOCUSED_GLOSSARY: <RELEVANT_TABLE_COLUMN_VALUE_MAP>
OPTIONAL_SQL_ALIGNMENT_CONTEXT: <GOLD_SQL_OR_OPERATOR_SIGNATURE>

OUTPUT SCHEMA
{
  "question": "<TRANSLATED_QUESTION>",
  "used_mapping_ids": ["<ID>"]
}

```

Figure 11: Prompt for question translation.

```

SYSTEM
Audit whether a translated analytical question preserves the source intent
and remains answered by the translated SQL. Do not solve the query.

CHECK
1. Requested output entities and fields.
2. All filters, values, comparison directions, and negations.
3. Aggregation, grouping, DISTINCT semantics, ordering, ranking, and limits.
4. Time windows, boundary inclusivity, units, and numerical scale.
5. Consistency with the supplied schema/value glossary.

INPUT
SOURCE_QUESTION: <SOURCE_QUESTION>
TRANSLATED_QUESTION: <TRANSLATED_QUESTION>
SOURCE_SQL: <SOURCE_SQL>
TRANSLATED_SQL: <TRANSLATED_SQL>
FOCUSED_GLOSSARY: <RELEVANT_MAP>

OUTPUT SCHEMA
{
  "status": "pass | repair | reject",
  "issues": [
    { "type": "<ISSUE_TYPE>", "description": "<DESCRIPTION>" },
  ],
  "revised_question": "<ONLY_IF_REPAIR>"
}

```

Figure 12: Prompt for question–SQL alignment.

A.1.7 Validation Gates and Failure Handling. Validation proceeds from inexpensive structural checks to semantic checks:

- (1) **Map validation** checks coverage, scope, protected identities, and table/column target-name collisions.
- (2) **Database validation** checks that mapped entities exist, dependent definitions resolve, and integrity constraints can be applied.
- (3) **Workload validation** parses σ_c , resolves every identifier, executes it on $\mathcal{D}_c$, and compares the result with $\tau_M(\text{Exec}(\mathcal{D}_0, \sigma_0))$. Comparison preserves duplicate multiplicity; row order is enforced when specified by the query.
- (4) **Question validation** uses an LLM judge to check that q_c remains answerable by σ_c and preserves its analytical intent.

Figure 12 instantiates the final semantic audit.

A tuple advances only when all gates pass. A failed map is regenerated or edited before any rewrite. A failed SQL is repaired against

the same map and re-executed. A repaired question is subjected to the alignment prompt again. Cases that cannot be made execution-aligned and semantically faithful are discarded. The accepted stage artifact contains $(\mathcal{D}_c, q_c, \sigma_c)$, the replacement map, synchronized metadata, and validation status; the final question–workspace answerability check is performed later by the expert review process in Section 4.4.

A.2 Constraint-Aware Relational Sampling Details

This appendix expands the relational sampling stage in Section 4.2. The procedure constructs a new task-local database instance rather than approximating the answer obtained from the source database. Accordingly, the source and sampled answers may differ in their entities, multiplicities, aggregate values, and ordering. Executability and task validity are checked on the sampled instance itself.

A.2.1 Sampling Contract. The input is the verified cross-language tuple $T_c = (\mathcal{D}_c, q_c, \sigma_c)$. Let $\mathcal{T}_c$ be the complete table inventory of $\mathcal{D}_c$, and let $\mathcal{D}_c[R]$ denote the rows of table R . A sampling policy

$$\psi = (z, \{b_R : R \in \mathcal{T}_c\}, K, R_a, \mathcal{E}^+) \quad (12)$$

contains a base random seed z , a soft row budget b_R for each table, a maximum number of attempts K , an optional anchor table R_a , and schema relationships $\mathcal{E}^+$ not declared in the source database. The latter covers, for example, known key pairs in databases with incomplete foreign-key metadata. Budgets bound ordinary random additions; protected rows and rows introduced by relational closure take precedence and may exceed them. We use $K = 3$ for the benchmark construction.

Every sampled instance preserves the complete table and column inventory:

$$\text{Tables}(\mathcal{D}_s) = \mathcal{T}_c, \quad \text{Schema}(\mathcal{D}_s[R]) = \text{Schema}(\mathcal{D}_c[R]). \quad (13)$$

Thus, sampling changes table contents but does not use the source SQL to remove tables or columns. Empty source tables remain valid empty tables with their schema intact.

A.2.2 Safeguard Extraction. We form the safeguard set $C_s = C_{\text{schema}} \cup C_{\text{query}}$. The schema component is represented as a directed relationship graph $G_s = (\mathcal{T}_c, \mathcal{E}_s)$. Each edge records a child table and column, the corresponding parent table and column, and whether the relationship is declared or supplied by $\mathcal{E}^+$. Composite keys are retained as tuples rather than decomposed into independent column constraints.

The query component is extracted from a resolved SQL AST. Name resolution is performed separately inside each query block so that aliases, common-table expressions, and correlated subqueries do not create spurious bindings. We record four types of safeguard:

- (1) **Predicate bindings** associate a resolved column with a literal or literal set used by equality, membership, range, or pattern predicates. Literal types are preserved during matching.
- (2) **Query relationships** record the resolved column pairs in explicit and implicit joins, including multi-column join keys.
- (3) **Boundary bindings** retain values that define temporal or numerical intervals.
- (4) **Target bindings** identify explicitly named entities whose disappearance would change the referent of the question.

Unsupported expressions do not trigger string-based guessing. They are retained in the AST record and left to execution validation; samples for which the protected values or join endpoints cannot be resolved are marked for repair before sampling.

A.2.3 Anchor Rows and Soft Budgets. For each table R , the sampler first constructs an anchor set A_R . Rows matching protected equality, membership, or target bindings are inserted directly. For range and pattern predicates, matching rows are selected using the same typed operator as the SQL expression. When a predicate is attached to a nested query, its anchors remain scoped to the base relation resolved in that query block. When R_a is configured, its selected rows provide the starting keys for relationship propagation. Anchor selection therefore operates on database values, not surface-form occurrences in serialized rows.

The remaining capacity is filled by sampling within each table. For a table R whose anchors do not exhaust its budget b_R , the initial row set is

$$S_R^{(0)} = A_R \cup \text{Sample}(\mathcal{D}_c[R] \setminus A_R, b_R - |A_R|). \quad (14)$$

Here, $\text{Sample}(X, n)$ returns up to n rows from X , so small tables are retained in full. If the anchors already meet or exceed the budget, no additional rows are sampled. Tables not referenced by σ_c use the same rule with an empty anchor set.

A.2.4 Relational Closure. Independent table samples can contain a selected foreign key without its referenced row or leave a protected query join without any matched pair. We therefore augment the initial row sets by relational closure. For every selected child row, its non-null referenced key introduces the matching parent row. For query relationships, the sampler additionally retains matched rows along the protected join path from the current anchors. Newly introduced rows are placed back on the closure queue until no relationship adds a row. Since closure only adds rows drawn from the finite source instance, the procedure terminates even when the schema graph contains cycles.

One-to-many expansion is bounded for ordinary random rows. Anchor-derived and query-path matches have priority; optional matches are drawn with the relationship-specific random stream until the relevant soft budget is reached. Missing referenced keys, type-incompatible join columns, and an empty protected join are recorded as validation errors rather than repaired by fabricating records.

Algorithm 4 summarizes the complete procedure.

A.2.5 Materialization and Validation. Materialization creates a fresh SQLite database, recreates every source table with its column types and key declarations, inserts the selected rows, and rebuilds applicable indexes and views. Inserts occur inside a transaction with foreign-key checking enabled at validation time. The resulting database is then checked in two stages.

Structural validation. We verify table and column inventory, declared primary-key uniqueness, foreign-key consistency, row serializability, and the presence of protected bindings and query-path matches. We also compare per-table row counts against the recorded selected-row sets; this catches silent insertion loss caused by type conversion or duplicate handling.

Algorithm 4 Constraint-aware relational sampling**Require:** Verified tuple $(\mathcal{D}_c, q_c, \sigma_c)$, policy ψ **Ensure:** Sampled database $\mathcal{D}_s$, reference result $\mathcal{Y}_s$, ledger L_s

```

1:  $C_{\text{schema}} \leftarrow \text{SCHEMA\_RELATIONS}(\mathcal{D}_c, \mathcal{E}^+)$ 
2:  $C_{\text{query}} \leftarrow \text{QUERY\_SAFEGUARDS}(\sigma_c, \mathcal{D}_c)$ 
3:  $C_s \leftarrow C_{\text{schema}} \cup C_{\text{query}}$ 
4: for attempt  $k = 1, \dots, K$  do
5:    $A \leftarrow \text{ANCHORROWS}(\mathcal{D}_c, C_{\text{query}}, R_a, z, k)$ 
6:    $S \leftarrow \text{SEEDEDTABLESAMPLES}(\mathcal{D}_c, A, \psi, k)$ 
7:    $S \leftarrow \text{RELATIONALCLOSURE}(S, C_s)$ 
8:    $\mathcal{D}_s \leftarrow \text{MATERIALIZESALLSCHEMAS}(\mathcal{D}_c, S)$ 
9:    $v_s \leftarrow \text{VALIDATESTRUCTURE}(\mathcal{D}_s, C_{\text{schema}})$ 
10:  if  $v_s = \text{pass}$  then
11:     $(e_s, \mathcal{Y}_s) \leftarrow \text{EXECUTE}(\mathcal{D}_s, \sigma_c)$ 
12:     $v_q \leftarrow \text{VALIDATETASK}(e_s, \mathcal{Y}_s, \sigma_c, C_{\text{query}})$ 
13:    if  $v_q = \text{pass}$  then
14:       $L_s \leftarrow \text{BUILDLEDGER}(\mathcal{D}_s, \mathcal{Y}_s, \psi, C_s, k)$ 
15:      return  $\mathcal{D}_s, \mathcal{Y}_s, L_s$ 
16:    end if
17:  end if
18: end for
19: return reject

```

Execution validation. The transformed SQL σ_c is parsed and executed on $\mathcal{D}_s$. Successful execution defines the candidate answer $\mathcal{Y}_s = \text{Exec}(\mathcal{D}_s, \sigma_c)$; no equality test against the source answer is applied. We check the returned arity against the resolved projection and record query-signature diagnostics for conditions such as a broken join, a null-only aggregate, or fewer candidates than a requested ranking. These conditions are not universal rejection rules: an empty or null-valued answer can be semantically correct. An unexpected empty result caused by the loss of protected predicates or join matches fails the attempt; ambiguous cases are forwarded to the expert review stage.

A.2.6 Retry, Repair, and Provenance. A failed attempt is retried with a deterministically derived seed. Failures caused by insufficient optional coverage are handled by resampling; failures caused by unresolved relationships, missing protected values, or unsuitable budgets require configuration repair before another attempt. A task is discarded when it exhausts the configured attempt budget or cannot produce an executable and meaningful relational instance.

For every accepted task, the internal sampling ledger stores the source task and database identifiers, SQL hash, policy and attempt seed, per-table soft budgets, extracted safeguards, added relationship pairs, row counts before and after sampling, retry history, intermediate-database hash, execution status, and candidate-answer hash. The ledger is used to reproduce construction and audit later repairs; it is not included in the agent-visible workspace.

A.3 Modality Routing & Artifact Rendering Details

This appendix expands Section 4.3. We first specify query-independent base routing and verify each rendered relation against the task-local intermediate database. We then detail fact-grounded document rendering and the separate query-conditioned video branch, and trace both procedures with released tasks. The algorithms describe the construction interfaces; renderer-specific

Algorithm 5 Query-independent base routing and materialization**Require:** Sampled database $\mathcal{D}_s$, policy π_r , seed z_r **Require:** Base renderer set $\mathcal{F}_{\text{base}}$, coverage state H **Require:** Fallback order O_f **Ensure:** Base workspace $\mathcal{W}_{\text{base}}$

```

1:  $\mathcal{W}_{\text{base}} \leftarrow \emptyset$ 
2: for all  $R \in \text{SEEDEDORDER}(\mathcal{D}_s, z_r)$  do
3:    $m_R \leftarrow \text{TABLEMETADATA}(R)$ 
4:    $E_R \leftarrow \text{COMPATIBLE}(m_R, \mathcal{F}_{\text{base}})$ 
5:    $U_R \leftarrow \text{ROUTE}(m_R, E_R, H, \pi_r, z_r)$ 
6:   if  $U_R = \emptyset$  then
7:      $U_R \leftarrow \{\text{FIRST}(E_R)\}$ 
8:   end if
9:   for all  $f \in \text{WITHFALLBACKS}(U_R, E_R, O_f)$  do
10:     $a \leftarrow \text{RENDER}(R, f)$ 
11:    if  $\text{ROUNDTripVALID}(a, R, f)$  then
12:       $\mathcal{W}_{\text{base}} \leftarrow \mathcal{W}_{\text{base}} \cup \{a\}$ 
13:       $H[f] \leftarrow H[f] + 1$ 
14:       $\text{MARKREPRESENTED}(R)$ 
15:      if  $\text{ASSIGNMENTSCOMPLETE}(R, U_R)$  then
16:        break
17:      end if
18:    end if
19:   end for
20:   if  $\neg \text{REPRESENTED}(R)$  then
21:     return reject
22:   end if
23: end for
24: return  $\mathcal{W}_{\text{base}}$ 

```

templates and visual themes can change without modifying their grounding and validation contracts.

A.3.1 Query-Independent Base Routing and Materialization.

Base-renderer assignment. For every sampled table $R \in \mathcal{D}_s$, the base router extracts a descriptor m_R containing its schema, column types, key declarations, row and column counts, missing-value profile, and serializability constraints. It does not inspect the question, task SQL, or candidate answer. Its renderer set is

$$\mathcal{F}_{\text{base}} = \{\text{CSV}, \text{JSON}, \text{SQLite}, \text{Markdown}, \text{PDF}\}, \quad (15)$$

Video is not a member of $\mathcal{F}_{\text{base}}$: it operates at the task level and may depend on the question, SQL structure, and candidate answer rather than materializing a single table R . The base policy first removes renderers that cannot faithfully encode m_R . Flat tables can be written as CSV or record-oriented JSON; relational groups with declared keys can be retained in SQLite; and tables selected for long-form rendering are passed to the Markdown/PDF document generator. A fixed seed breaks ties among eligible choices, while batch-level counters favor underrepresented modalities. These counters balance assignments across the collection; they do not prescribe the number of files, artifact sizes, or the proportions of long and wide tables in an individual workspace.

Algorithm 5 gives the normalized procedure. A table may be assigned to more than one renderer, and every table must have at least one successful base representation. When a preferred renderer fails its round-trip check, the router tries the next compatible choice; failure of all choices rejects the materialization attempt.

Serialization contract. Each structured renderer is paired with a parser that reconstructs a canonical relation. The comparison covers the column inventory, typed cell values, null positions, duplicate-row multiplicity, and row count. CSV uses quoted fields and an explicit schema side record during construction; JSON is written as a list of records with stable keys; and SQLite recreates declared types, primary keys, and foreign keys before inserting rows. File order is not treated as relational meaning unless an order-bearing field is itself part of the table. For a renderer f , acceptance requires

$$\text{Canon}(\text{Parse}_f(\text{Render}_f(R))) = \text{Canon}(R), \quad (16)$$

where canonicalization normalizes physical encodings while retaining data types and row multiplicity. The construction ledger records the table, renderer, seed, file hash, row count, and validation result. This information supports reproduction and repair but is not placed in the released workspace.

A.3.2 Fact-Grounded Table-to-Document Rendering.

Document plan. Document rendering converts a complete sampled table into a long-form report without placing the full table in a prompt at once. The LLM-based planner first infers a domain-appropriate genre, such as a clinical event ledger, financial audit, or operational briefing. It then chooses anchor columns that identify the entity or record. Declared keys receive priority; otherwise, high-uniqueness identifiers and stable entity names are used. The remaining columns are grouped by semantic role, and rows are partitioned to respect the generation budget. A small table is represented by a single row batch and attribute cluster, making one-pass generation a special case of the same procedure. For larger tables, the plan can emphasize rows (a horizontal pass), attribute clusters (a vertical pass), or alternate between them. In each case, the generation unit reduces to anchors, a bounded row set, and one or more attribute clusters.

Every generation block contains (i) its row identifiers, (ii) the anchor columns repeated from those rows, and (iii) one attribute cluster. Repeating anchors allows sections generated from distant attribute clusters to remain joinable after assembly. Each non-null source cell is assigned an internal cell ID, and each block carries the set of cell IDs that it must express. Null-valued fields follow the plan's explicit policy: either state that the field is unavailable or omit it without inventing a replacement.

Grounded generation. Algorithm 6 shows the generation and checking loop. The model may vary discourse structure, connective text, and non-evidential background detail, but protected identifiers, numbers, dates, units, and categorical values must remain recoverable with their source precision. When contrastive or corrective prose is used for stress testing, the source value must be identified unambiguously as the final record value. A failed block is regenerated from the validator feedback rather than silently removed.

Controlled document complexity. Three controls vary extraction difficulty without changing the sampled table. Recursive detailing expands a block into successively more specific report sections. The null policy varies whether missing fields are stated or omitted. Finally, a configurable subset of blocks receives domain-relevant

Algorithm 6 Fact-grounded table-to-document rendering

Require: Sampled table X_R , schema metadata m_R

Require: Token budget B , retry limit K , output format f

Ensure: Markdown or PDF artifact d_R

```

1:  $(\eta_R, \mathcal{K}_R, C_R, I_R) \leftarrow \text{PLANDOCUMENT}(X_R, m_R, B)$ 
2:  $\mathcal{B}_R \leftarrow \text{BUILDBLOCKS}(X_R, \mathcal{K}_R, C_R, I_R)$ 
3:  $\mathcal{G}_R \leftarrow \emptyset$ 
4: for all  $B_j \in \mathcal{B}_R$  do
5:   for  $k = 1$  to  $K$  do
6:      $g_j \leftarrow \text{GENERATESECTION}(B_j, \eta_R)$ 
7:      $e_j \leftarrow \text{VALIDATECELLS}(g_j, B_j)$ 
8:     if  $e_j = \emptyset$  then
9:       break
10:    end if
11:     $B_j \leftarrow \text{ATTACHFEEDBACK}(B_j, e_j)$ 
12:  end for
13:  if  $e_j \neq \emptyset$  then
14:    return reject
15:  end if
16:   $\mathcal{G}_R \leftarrow \mathcal{G}_R \cup \{g_j\}$ 
17: end for
18:  $d_R \leftarrow \text{ASSEMBLEMARKDOWN}(\mathcal{G}_R, \eta_R)$ 
19:  $\text{VALIDATECOVERAGE}(d_R, X_R)$ 
20: if  $f = \text{PDF}$  then
21:    $d_R \leftarrow \text{CONVERTANDCHECKPDF}(d_R)$ 
22: end if
23: return  $d_R$ 

```

narrative context, nearby non-answer facts, or a correction-style presentation. Such additions cannot replace a required source cell, alter a protected value, or introduce a competing final value for the same record and field.

The block validator combines exact and typed checks. Exact-token checks cover IDs, codes, protected strings, and categorical values. Numeric values are parsed with their units and compared at the recorded precision; dates and times are normalized before comparison; and ordinary text values are checked against the source cell or an approved surface-form map. The final coverage pass verifies that every required cell ID is linked to at least one document span and that no span assigns two incompatible values to the same record-field pair. PDF conversion is followed by text extraction and page decoding checks. Figure 13 gives the normalized block-level generation contract.

A.3.3 Document-Rendering Running Example. Figure 14 traces one row from the LABEVENTS document in released Task 310. The sampled row contains seven fields. The planner uses ROW_ID as the anchor and separates event context from the laboratory measurement. The two resulting blocks are therefore generated at different positions in a long clinical ledger but can be joined through event 142456.

All seven source cells are aligned to the two spans, with the anchor appearing in both. Recovering the measurement as a structured record requires associating fields across separated narrative sections through the exact record identifier while ignoring surrounding clinical and operational prose.

A.3.4 Query-Conditioned Video Rendering.

```

SYSTEM
Write one section of a realistic <DOCUMENT_GENRE> from the supplied
table block. The table block is the sole source of record facts.

GROUNDING RULES
1. Express every REQUIRED_CELL_ID with its exact entity association.
2. Preserve identifiers, categorical values, numeric precision, dates,
   times, signs, and units. Do not merge values from different rows.
3. Follow NULL_POLICY. Never infer a missing value.
4. You may add connective prose or domain-neutral scene detail, but it
   must not create a competing value for any protected field.
5. Keep ANCHOR_FIELDS explicit so this section can be joined with other
   sections about the same records.
6. Return only the requested section and the cell-to-span alignment JSON.

INPUT
STYLE: <STYLE_SPECIFICATION>
ANCHOR_FIELDS: <ANCHOR_COLUMNS_AND_VALUES>
ATTRIBUTE_CLUSTER: <COLUMN_NAMES_TYPES_AND_DESCRIPTIONS>
ROWS: <TYPED_VALUES_WITH_CELL_IDS>
REQUIRED_CELL_IDS: <CELL_IDS>
NULL_POLICY: <EXPLICIT_OR_OMIT>

OUTPUT
{
  "section": "<MARKDOWN_SECTION>",
  "alignments": [
    { "cell_id": "<CELL_ID>", "surface": "<TEXT_SPAN>" }
  ]
}

```

Figure 13: Prompt for fact-grounded document generation.

Evidence selection. Video augmentation begins only after the sampled SQL has been executed. The selector parses comparison predicates, projected fields, aggregation and ordering operators, and the candidate tabular result. A predicate candidate is represented as a typed atom

$$e^{\text{pred}} = (t, c, o, v, \lambda, \gamma), \quad (17)$$

where t and c identify the source table and column, o is the operator, v is the typed condition value, λ is its aligned question span, and γ records display constraints such as precision and units. Null tests, join keys, grouping operators, unstable relative-time conditions, and projection-only fields are excluded from predicate abstraction. An answer atom

$$e^{\text{ans}} = (r, c, v, \gamma) \quad (18)$$

identifies one result row, output field, typed value, and display contract. Answer-evidence rendering is used only when the result is compact enough for its required atoms to remain legible across scenes.

Two rendering strategies. In *predicate abstraction*, one or more stable predicate atoms are removed from the explicit question and expressed through a business scene, such as a configuration panel, time window, or eligibility rule. The remaining workspace still contains the records on which the recovered predicate must be applied. In *answer-evidence rendering*, result atoms are distributed across tables, charts, cards, or temporally separated views. Direct copies of the same answer-bearing source are withheld when they would make the video unnecessary. In both cases, distractors are drawn from nearby fields, entities, periods, or boundary examples; they cannot change the selected atom or create a second valid answer.

Algorithm 7 Query-conditioned video augmentation

Require: Question q_c , SQL σ_c , answer $\mathcal{Y}_s$
Require: Base workspace $\mathcal{W}_{\text{base}}$, strategy h
Ensure: Adapted question q_r , workspace $\mathcal{W}$

- 1: $A \leftarrow \text{EXTRACTTYPEDATOMS}(\text{AST}(\sigma_c), \mathcal{Y}_s, q_c)$
- 2: $\mathcal{E}_v \leftarrow \text{SELECTATOMS}(A, h)$
- 3: **if** $\mathcal{E}_v = \emptyset$ **then**
- 4: **return reject**
- 5: **end if**
- 6: $S_v \leftarrow \text{PLANSTORYBOARD}(\mathcal{E}_v, h)$
- 7: $\text{VALIDATEATOMCOVERAGE}(S_v, \mathcal{E}_v)$
- 8: $C_v \leftarrow \text{COMPILECOMPOSITION}(S_v)$
- 9: $a_v \leftarrow \text{SYNTHESIZENARRATION}(S_v)$
- 10: $v \leftarrow \text{RENDERVIDEO}(C_v, a_v)$
- 11: $q_r \leftarrow \text{REWRITEALIGNEDSPANS}(q_c, \mathcal{E}_v, h)$
- 12: $\mathcal{W} \leftarrow \text{INTEGRATE}(\mathcal{W}_{\text{base}}, v, \mathcal{E}_v, h)$
- 13: $\text{VALIDATEVIDEOTASK}(q_r, \mathcal{W}, v, \mathcal{E}_v)$
- 14: **return** $q_r, \mathcal{W}$

Storyboard and rendering. Each selected atom receives a stable evidence ID. A storyboard scene lists its duration, evidence IDs, visual component, exact data constants, visible text, and narration. The planner can decide how an atom is communicated, but the constants in its component specification are filled from the typed evidence record. Supporting records for boundary cases and distractors are queried from the sampled database and retain their source table and row identifiers. The storyboard is compiled into a task-specific React/Remotion composition. Narration is synthesized separately, and measured audio duration is used to set scene timing. Compilation or rendering failures are repaired at the component level and rendered again in task isolation.

Algorithm 7 summarizes the branch. Question rewriting operates over the aligned span λ : it replaces only the selected condition or answer reference with a description that points to the video. The requested output fields, aggregation, comparison, ordering, units, and temporal scope remain unchanged. The question is rejected when the selected span cannot be removed without changing those semantics.

The normalized storyboard-planning contract appears in Figure 15.

A.3.5 Video-Rendering Running Example.

Predicate-abstraction example. Released Task 193 asks which equity-freeze records meet the “Major Share Freeze Alert” configured in the video and requests shareholder, involved shares, start date, and end date. The underlying record filter contains two typed predicate atoms: security code 600180 and $\text{PCTOfTotalShares} \geq 0.05$. Rather than placing these constants in the question, the video identifies the monitored security and later displays a configuration panel whose field is PCTOfTotalShares , direction is $\geq$, and cutoff is 5.00%. Table 4 follows these atoms into the released task, while Figure 16(a)–(c) shows their visual carriers in the rendered video.

Solving this task combines the entity scope and predicate from the video with the freeze records in SQLite and the output projection in the question. Neither the percentage field nor the 5.00% cutoff is part of the requested output.

Document plan: clinical event ledger anchor $\mathcal{K}_R = \{\text{ROW_ID}\}$ context cluster C_1 measurement cluster C_2					
Sampled row		$\Rightarrow$	Grounded blocks	$\Rightarrow$	Released document spans
- ROW_ID: 142456 - SUBJECT_ID: 23070 - HADM_ID: 127721 - CHARTTIME: 2105-12-09 20:58:00 - ITEMID: 50954 - VALUENUM: 151.0 - VALUEUOM: iu/l		$\Rightarrow$	B_1: event 142456; patient 23070; admission 127721; timestamp 2105-12-09 20:58:00. B_2: event 142456; item 50954; result 151.0 iu/l. The anchor is repeated so that independently generated sections remain joinable.	$\Rightarrow$	“Regarding lab event 142456, this record pertains to patient 23070 during hospital admission 127721 ... December 9th, 2105, at 20:58:00.” “For event 142456, the test ordered under item code 50954 ... yielded a result of 151.0 ... iu/l.”
					anchor / record identity event context laboratory measurement

Figure 14: Source-to-document alignment for one sampled LABEVENTS row in Task 310. Colors preserve field-group provenance from the source row through grounded generation blocks to mentions in the released Markdown document; ellipses shorten the displayed spans.

Table 4: Predicate-abstraction trace for released Task 193. Values in the application row are read from the task-local SQLite artifact; the output table omits the trigger percentage because it is not requested.

Stage	Instantiated task content
Selected atoms	$e_1 = (\text{SecuCode}, =, 600180)$; $e_2 = (\text{PCTOfTotalShares}, \geq, 0.05)$. The latter retains the display form 5.00% and the comparison direction.
Video scenes	An early security-monitoring scene establishes code 600180. Boundary examples contrast records below and above the cutoff. A later configuration scene displays trigger field PCTOfTotalShares, direction $\geq$, and “Alert Cutoff = 5.00%.”
Adapted question	“According to the Major Share Freeze Alert configuration defined in the video, which equity-freeze records of the monitored security meet the alert threshold? Return the shareholder, involved shares, start date, and end date.”
Application workspace	to For security 600180, the Shen Renrong record has involved shares 15,000,000 and PCTOfTotalShares=0.0598, so it passes. A neighboring record for Zhengzhou R.M.T. Supply Chain Co. has involved shares 21,000,000 and percentage 0.024, so it fails. The agent must recover the predicate from the video before filtering <code>lc_sharefp</code> in SQLite.
Reference row	The passing example contributes (Shen Renrong, 15000000, 2012-04-27, 2012-07-24) to the four-column reference table; all other passing records are returned under the same schema.

Answer-evidence companion. Task 82 illustrates the other strategy. Its video presents two transfer rows for company 79959 and a descending sort on pre-transfer ownership. The rows shown are

$$\begin{aligned}
 & (79959, 0.4787, 0.4787, 2018-03-15), \\
 & (79959, 0.4664, 0.4640, 2021-06-11),
 \end{aligned} \tag{19}$$

with columns for company code, ownership before transfer, ownership after transfer, and transaction date. The released question asks which record has the higher pre-transfer percentage. The required output atom is therefore the first row, while the second row supplies a same-entity comparison. The source transfer relation is not duplicated as an agent-visible structured artifact; the remaining workspace contains related company data and natural distractors. This task can be solved only after reading the two percentages, associating them with the correct rows, and applying the requested comparison. Figure 16(d) shows the corresponding ranked view.

A.3.6 Artifact Validation and Failure Handling. Validation is applied at three levels. First, renderer checks compare CSV, JSON, and SQLite round trips with the sampled relations using Equation 16. Second, document checks verify required cell coverage, protected tokens, typed numeric/date/unit fidelity, null handling, document

decoding, and the absence of incompatible claims for one record-field pair. Third, video checks verify the evidence schema, atom-to-scene coverage, component constants, question-span replacement, compilation, audio presence, file decoding, duration, and sampled frames.

A structural failure triggers deterministic fallback, block regeneration, or component repair as appropriate. A semantic failure—for example, a misstated document value, a missing comparison direction, an unreadable video constant, or a question that still exposes relocated evidence—returns the artifact to generation with a typed error report. Tasks that cannot satisfy the contract within the retry budget are rejected. Accepted artifacts then enter Human Review & Task Repair in Section 4.4, where reviewers inspect the complete question, workspace, and reference answer together.

A.4 Construction Models and Cost

Table 5 reports the LLM configurations and mean API cost per task for the model-based construction components.

The document cost includes generation and model-based validation. The video cost is estimated from sampled successful video-generation records. Reported costs cover model API usage and

```

SYSTEM
Design a realistic data-video storyboard that communicates every supplied
evidence atom without changing its value or analytical role.

RULES
1. Every EVIDENCE_ID must appear in at least one scene and must retain its
typed value, operator, precision, unit, and entity association.
2. Predicate abstraction must communicate both comparison direction and
threshold. Answer evidence must remain readable but need not be adjacent.
3. Use only supplied records for data-bearing distractors. Do not invent an
alternative rule or a second valid answer.
4. Narration should support the business scenario; do not read out all
evidence when doing so would make visual structure unnecessary.
5. Return JSON only. Every visible constant must identify its source atom.

INPUT
TASK_STRATEGY: <PREDICATE_ABSTRACTION_OR_ANSWER_EVIDENCE>
EVIDENCE_ATOMS: <TYPED_ATOMS_WITH_STABLE_IDS>
SUPPORTING_RECORDS: <BOUNDARY_AND_DISTRACTOR_RECORDS>
VISUAL_COMPONENT_LIBRARY: <AVAILABLE_COMPONENTS>

OUTPUT
{
  "scenes": [
    {
      "scene_id": "...", "duration_hint": "...",
      "evidence_ids": [...], "component": "...",
      "data_constants": [{"atom_id": "...", "value": "..."}],
      "visible_text": [...], "narration": "..."
    }
  ]
}

```

Figure 15: Prompt for evidence-grounded storyboard planning.

Table 5: LLM configurations and API costs for benchmark construction.

Component	Model and role	Mean USD/-task
Cross-language transformation	Qwen-Plus (generation)	\$0.000189
Document rendering	Gemini 2.5 Pro (generation); GPT-4o (validation)	\$0.92
Video rendering	Claude Sonnet 4.6 (generation)	~\$0.58

exclude expert review, local rendering, storage, and other infrastructure.

A.5 Human Review & Task Repair Details

Human review operates on a complete candidate task version comprising its question, workspace, and reference result. It begins only after the structural and artifact-level checks in Appendix A.3. Those checks can establish that files decode, database relations round-trip, and protected values are preserved; the expert protocol determines whether the assembled task has a clear user intent, sufficient evidence, a unique answer, and suitable evaluation semantics.

A.5.1 Staffing, Assignment, and Blinding. The review pool comprises 11 domain experts with recorded domain and data-analysis expertise. Each task is assigned to at least two reviewers. Assignments are balanced across the active pool and can be adjusted for domain coverage before review begins. A pending assignment may be replaced, but reviewer membership is fixed once an independent answer has been submitted.

Reviewers, repairers, and administrators have separate roles. Reviewers solve and verify assigned tasks; repairers prepare a new task version after a disagreement; administrators manage assignments

and record the final accept-or-remove decision. During independent solving, a reviewer cannot access the source SQL, reference result, discussion history, or peer submissions. Peer judgments become visible only after that reviewer has locked both stages of their own review.

A.5.2 Two-Stage Review Record. Table 6 summarizes the information boundary and the record produced at each stage. Phase 1 requires a rectangular table with a header; a header-only table is valid when the correct answer is empty. Every submission must include at least one evidence reference. An evidence reference contains an artifact path, its modality, a row, key, section, page, or timestamp anchor, the supported fact, and its analytical role such as filter, join, projection, aggregation, ordering, or unit interpretation. The reviewer also records the interpreted grain, predicates, aggregation, ordering, units, null and duplicate semantics, issue flags, and confidence. The server validates referenced paths against the current task version and makes the submitted result immutable.

Phase 2 exposes the reference result. The reviewer assigns a gold verdict from MATCH, MISMATCH, and UNSURE, together with one disposition: PASS, repair the question, workspace, gold, or configuration, repair multiple components, or remove the task. Each reviewer independently authors a configuration covering the semantic type of every output column, any numeric precision and percentage convention, and whether row order is required. The configuration is derived from the question and verified gold rather than tuned after observing model predictions.

A.5.3 Consensus, Adjudication, and Versioned Repair. A task version reaches consensus only when every assigned reviewer has completed both review stages, marked the gold as MATCH, selected Pass, and submitted the same canonical evaluation configuration. Missing verifications never count as agreement. Consequently, reviewers who agree on all gold values still enter adjudication when their type, precision, or ordering configurations differ.

Adjudication is evidence based rather than a majority vote. Reviewers identify the conflicting claim and cite its location in the released workspace. The resulting repair record states its reason and identifies which components change: the question, workspace artifacts, reference result, evaluation configuration, or a combination of them. Repairs follow a minimal-change policy, retaining unaffected artifacts and semantics. Finalizing a repair preserves the previous version, activates a new one, and reassigns the same reviewers. The repaired version is independently rechecked before discussion resumes, and each reviewer submits a new verification and configuration for that version. Discussion, repair, and recheck repeat until the consensus conditions above hold. A task that cannot be repaired into an unambiguous and releasable instance is removed with an explicit reason.

A.5.4 Review Dimensions and Issue Taxonomy. Reviewers assess structural integrity, evidence sufficiency, answer uniqueness, gold correctness, cross-modal consistency, artifact fidelity, video necessity, question quality, evaluation validity, and release suitability. Table 7 groups the structured issue flags used to make these judgments comparable across tasks. Multiple flags may be attached to one review; free-text rationale and evidence anchors preserve the concrete claim behind each flag.

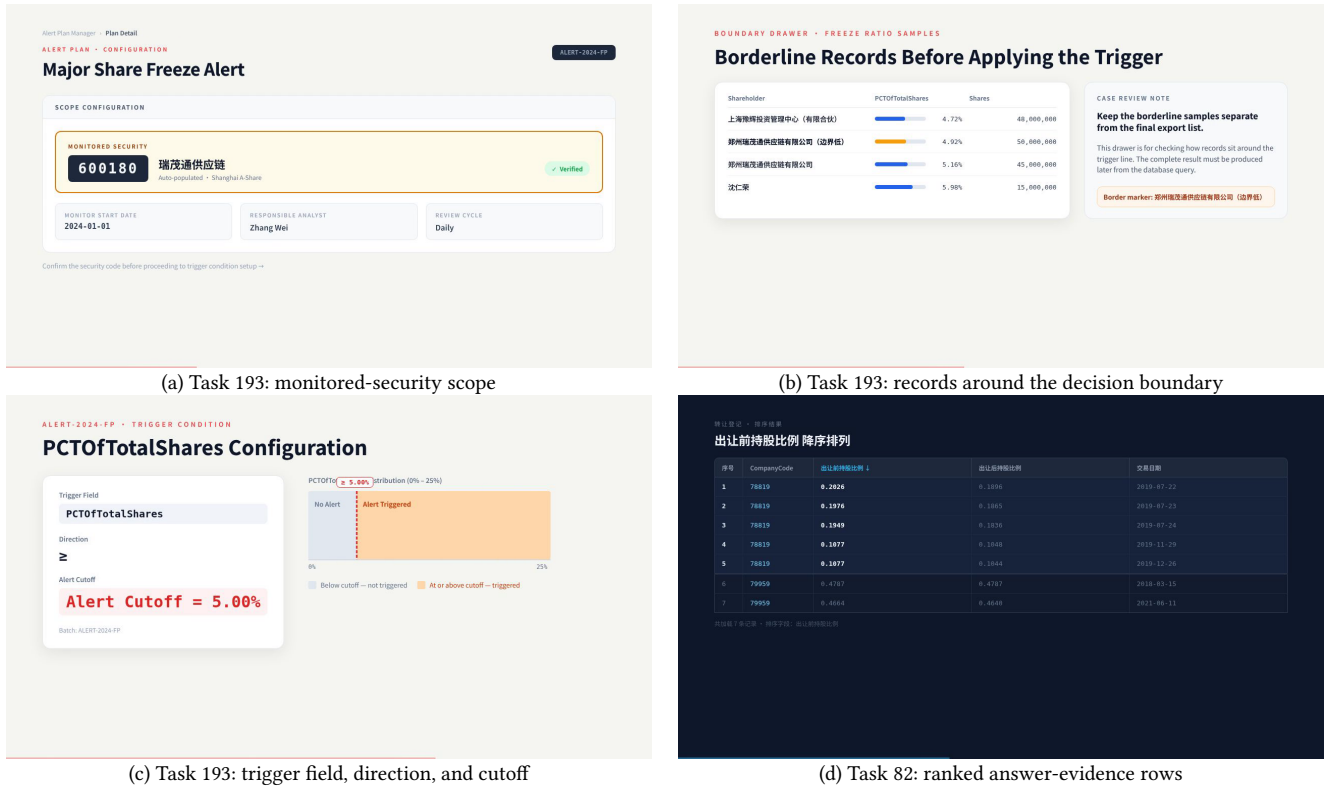


Figure 16: Representative frames from the released task videos. Task 193 distributes its security scope and predicate specification across separate scenes: panel (a) identifies the monitored security, panel (b) supplies boundary records, and panel (c) defines the trigger. Panel (d) shows the two same-company transfer records used by the answer-evidence strategy in Task 82.

Table 6: Information boundaries and required records in the expert-review protocol. Diagnostic table comparison is shown only after the independent answer has been locked and is not used as the official benchmark evaluator.

Stage	Visible to the reviewer	Required record
Blind independent solving	Final question and complete task-local workspace	Candidate result; artifact-level evidence anchors; answer semantics; issue flags; five-point confidence; optional rationale
Gold verification and configuration authoring	Locked candidate result, reference result, and diagnostic table difference	Gold verdict; release disposition; independently authored column types, numeric comparison rules, units, percent handling, and row-order flag; issue flags and rationale
Adjudication and recheck	Cross-review summaries and discussion after all participating judgments are locked; repaired version during independent recheck	Evidence-grounded resolution; repair scope and reason; new-version review or explicit removal reason

A.5.5 Audit Trail and Review Interface. The review system records assignments, independent submissions, verifications, state transitions, discussions, repair drafts and artifact replacements, version activation, acceptance, and removal as append-only events with actor, task version, UTC timestamp, and event payload. Base benchmark files remain read only; repaired context files are stored as versioned overlays. The resulting record supports agreement, issue, repair, and post-repair acceptance statistics without reconstructing decisions from the released files.

The browser console jointly presents the question, modality-filtered workspace, native artifact viewers, evidence form, result comparison, configuration editor, and discussion history. Figure 17

shows an illustrative session for released Task 193. The reviewer recovers an alert predicate from video, applies it to SQLite, and records both evidence anchors before the reference result is unlocked.

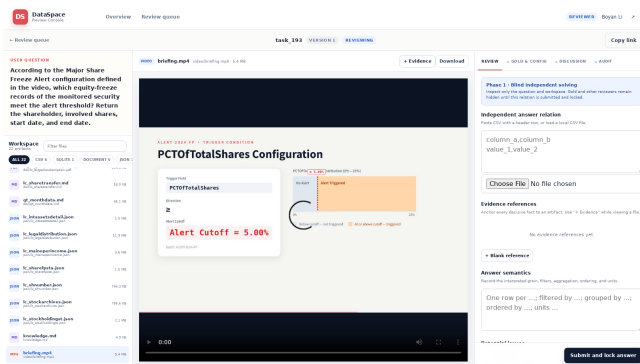
A.6 Evaluation Protocol Details

This appendix expands the evaluation protocol in Section 4.5. It specifies the frozen task configuration, canonicalization rules, joint column alignment, and tabular-result matching.

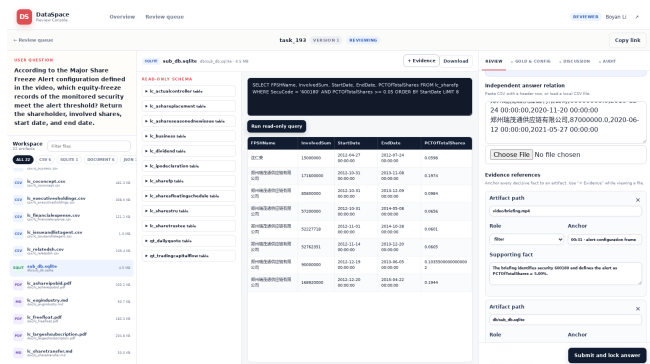
A.6.1 Frozen Per-Task Configuration. Each task configuration contains a schema version, the task identifier, an ordering flag, and

Table 7: Issue taxonomy used during independent solving and gold/configuration verification. The repair surface is selected separately, so one issue can require changes to multiple task components.

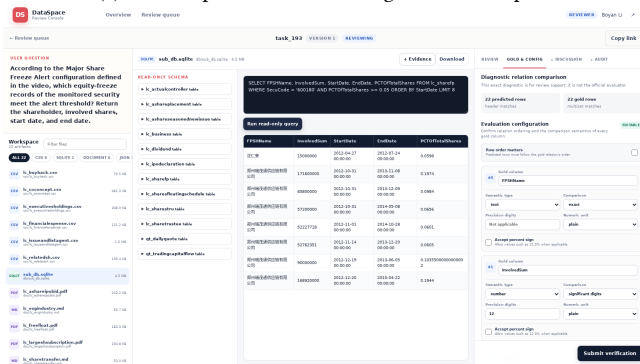
Category	Covered issues	Typical review question
Access and evidence	Parsing failure, missing evidence, or mutually conflicting evidence	Can every decisive fact be accessed and anchored in the workspace?
Question semantics	Ambiguous grain, filter, join, unit, time scope, ordering, null handling, or duplicate handling	Does the request determine one complete tabular result without relying on hidden assumptions?
Reference result	Question-gold mismatch and value, shape, or order errors	Are all and only the requested rows and columns returned with correct values?
Artifact fidelity	Document or video fact error, video leakage or non-necessity, translation drift, and cross-modal inconsistency	Do rendered artifacts faithfully carry their assigned data and reasoning role?
Evaluation semantics	Incorrect column type, precision, unit, percentage convention, or ordering flag	Does the configuration encode only distinctions required by the question?
Release suitability	Privacy or licensing risk, near-duplicate task, or unsolvable task	Can the task be released and evaluated without ambiguity or prohibited data?



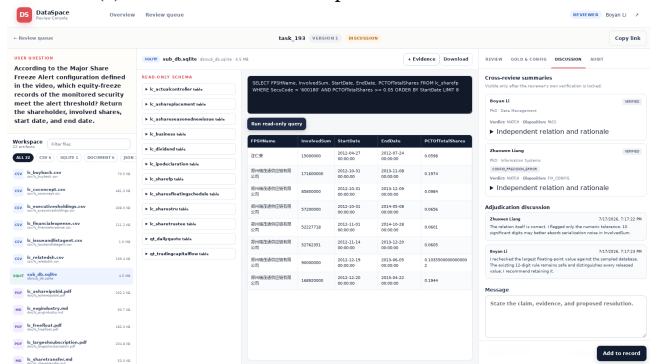
(a) Blind inspection of the heterogeneous workspace



(b) Evidence-anchored independent result submission



(c) Gold verification and evaluation-config authoring



(d) Cross-review comparison and recorded adjudication

Figure 17: Illustrative end-to-end reviewer trace in the human-review system. Panel (a) shows the blind phase, in which the question and complete workspace are visible while gold and peer reviews remain locked. Panel (b) links the candidate result to a video timestamp and filtered SQLite rows. Panel (c) unlocks result comparison and supports independent authoring of per-column type and precision settings after answer submission. Panel (d) exposes both completed reviews and records a configuration disagreement for adjudication.

one entry for every reference column. Column entries are indexed in reference order and record a human-auditable reference name and semantic type. Numeric entries additionally specify an integer, decimal-place, or significant-digit comparison rule; a unit convention and permission to use a percent sign are included when

relevant. The reference name is retained for auditing but is never compared with a prediction header.

Figure 18 gives the released configuration for Task 120. The task requests region, period-end date, and per-capita GDP in the order presented by the source. Accordingly, its rows are order sensitive,

```

{
  "schema_version": "1.0",
  "task_id": "task_120",
  "order_sensitive": true,
  "columns": [
    {
      "gold_index": 0,
      "gold_name": "Region",
      "type": "text"
    },
    {
      "gold_index": 1,
      "gold_name": "Period-end date",
      "type": "datetime"
    },
    {
      "gold_index": 2,
      "gold_name": "GDP per capita",
      "type": "number",
      "comparison": {
        "mode": "decimal_places",
        "digits": 1
      }
    }
  ]
}

```

Figure 18: Frozen evaluation configuration for Task 120.

and its three columns are normalized as text, datetime, and a number rounded to one decimal place.

A.6.2 Canonicalization Rules.

Text and nulls. Text values are stripped of surrounding white-space, line endings are standardized, and Unicode is normalized to NFC; comparison otherwise remains case sensitive. Identifiers and ordinary strings share this type because both require exact lexical equality after normalization. An empty text cell is canonicalized as null. For non-text columns, empty cells and the case-insensitive tokens null, none, nan, nat, and <na> are treated as null. A null value matches only another null.

Numbers. Numbers are parsed as finite base-10 decimals; ordinary decimal notation, scientific notation, and valid comma thousands separators are accepted. The integer mode requires an integral value. The decimal_places and significant_digits modes round both reference and predicted values with round-half-up at the configured precision, after which their canonical decimal representations must be identical.

The numeric unit is plain, percentage_points, or fraction. A percent-marked prediction is accepted only when allow_percent_sign is enabled. Under percentage_points, 3.5% denotes 3.5; under fraction, it denotes 0.035. An unmarked value is interpreted directly in the configured unit.

Dates, datetimes, and Booleans. Dates and datetimes use ISO syntax. A date column also accepts a midnight datetime, while a non-midnight time is invalid. Timezone-aware values are converted to UTC before comparison. Boolean values accept 1/true/yes/y and their Chinese affirmative counterpart as true, and 0/false/no/n and the corresponding Chinese negative token as false; alphabetic tokens are case insensitive.

A.6.3 Joint Alignment and Tabular-Result Matching. Prediction headers are required for valid CSV serialization but do not participate in scoring. After checking that prediction and reference have the same shape, the evaluator considers every one-to-one

Algorithm 8 Task-level tabular-result evaluation

Require: Prediction P , reference G , configuration $c = (o, \{v_j\}_{j=1}^d)$

Ensure: Binary task score

```

1: if  $\neg \text{VALIDCSV}(P)$  then
2:   return 0
3: end if
4: if  $\text{SHAPE}(P) \neq \text{SHAPE}(G)$  then
5:   return 0
6: end if
7:  $G^* \leftarrow \text{CANON}(G, c)$ 
8: for all  $\pi \in \Pi_d$  do
9:    $P_\pi \leftarrow \text{REORDERCOLUMNS}(P, \pi)$ 
10:  if  $\neg \text{COMPATIBLE}(P_\pi, c)$  then
11:    continue
12:  end if
13:   $P^* \leftarrow \text{CANON}(P_\pi, c)$ 
14:  if  $\text{Rows}(P^*, o) = \text{Rows}(G^*, o)$  then
15:    return 1
16:  end if
17: end for
18: return 0

```

mapping from predicted columns to reference columns. A mapping remains eligible only when all cells in each predicted column can be canonicalized by the rule of its mapped reference column. The mapped columns are then reassembled into rows before comparison, so values from different prediction rows cannot be combined independently.

For an order-sensitive task, the canonical row sequences must be identical. Otherwise, rows are compared as multisets, retaining the multiplicity of every duplicate row. Algorithm 8 gives the task-level procedure used by the official evaluator.

A.7 Experimental Configurations

A.7.1 Backbones. Table 8 records the exact endpoints used in the controlled backbone comparison. The six models were publicly released within the four months preceding evaluation and were accessed through Vercel AI Gateway in July 2026. We leave reasoning effort and all unspecified sampling parameters at the provider default; each endpoint receives the same 32,768-token output ceiling.

A.7.2 Agent Harnesses. The complementary harness comparison fixes the endpoint to xiaomi/mimo-v2.5. Table 9 lists the pinned implementations. All model roles exposed by a harness, including subagents and summarization or compaction calls, are mapped to the same backbone and model fallback is disabled.

DATASPACE-AGENT alternates model responses with tool observations until the model submits an answer or reaches a resource limit. It exposes three actions: bash executes local data-processing commands, view_image returns a selected image to the multimodal backbone, and submit_answer validates and submits a tabular result as CSV. Its prompt specifies the task, tool interfaces, workspace location, and output contract, without task-specific source selection, modality routing, or analytical operators.

A.7.3 Specialized Data-Agent Compatibility. Table 10 records their status at the time of evaluation in July 2026. We require a system to be reproducibly runnable over the complete task directory and

Table 8: Backbones used in the controlled DATASPACE-AGENT comparison. Release denotes the month of public model availability.

Backbone	Gateway model identifier	Release	Access
Grok 4.5 [39]	xai/grok-4.5	2026-07	Proprietary
GPT-5.6 Sol [32]	openai/gpt-5.6-sol	2026-07	Proprietary
Kimi K3 [29]	moonshotai/kimi-k3	2026-07	Open-weight
MiMo-V2.5 [1]	xiaomi/mimo-v2.5	2026-04	Open-weight
Claude Sonnet 5 [3]	anthropic/claude-sonnet-5	2026-06	Proprietary
MiniMax M3 [28]	minimax/minimax-m3	2026-06	Open-weight

Table 9: Agent harnesses compared with MiMo-V2.5 fixed as the backbone.

Harness	Version	Execution interface
DATASPACE-AGENT	Ours	Terminal-style ReAct with shell, image, and submission actions
Smolagents [36]	1.26.0	Code and tool execution in an isolated container
Codex [31]	0.145.0	Native CLI through the Responses API
Claude Code [2]	2.1.217	Native CLI with task-local configuration and state
Grok Build [38]	0.2.106	Native CLI through the Chat Completions API

to support the benchmark’s multimodal input and tabular-output contract without replacing core components.

FDABench likewise reports reimplementing TAIJI, AOP, and AgenticData from their papers because the original systems were not open-sourced [48]. Using such reconstructions, adding a video-capable perception model, or rewriting a system’s output path would introduce substantial choices absent from the original systems and prevent a faithful comparison.

A.7.4 Runtime and Reproducibility. For the backbone comparison, each DATASPACE-AGENT run is limited to 60 model turns, 50 tool actions, 1,800 seconds of wall-clock time, and 180 seconds per shell command. Its isolated runtime receives 4 CPUs, 16 GiB of memory, and no network access. Full-benchmark jobs use eight-way task concurrency, which affects throughput but not per-task limits.

For the harness comparison, all systems receive a fresh session, a task-local home directory, and Data Workbench Runtime 1.0. The runtime supplies generic CSV, JSON, SQLite, Markdown, PDF, image/OCR, and video utilities, but no retrieval, schema-linking, document-QA, video-QA, or Text-to-SQL solver. Model-generated commands cannot access the network. The host-side controller can reach only the configured inference endpoint and never exposes its credential to generated code. Each harness receives the same 1,800-second wall-clock limit, task input, and CSV output contract; its internal action budget and context-management policy remain native to that harness. Prompts, configuration files, validated predictions, and raw execution traces are retained for release.

A.8 Additional Statistics and Results

A.8.1 Backbone Efficiency. Table 11 reports the complete efficiency values underlying Figure 5. Cached prompt tokens are subsets of input tokens, and reported reasoning tokens are subsets of output tokens; neither is added again to the total. Costs are computed using the providers’ official pricing at the time of evaluation.

Efficiency on unsuccessful tasks. For five of the six backbones, an incorrect task consumes between 1.2 and 3.2 times as many mean tokens as a correct task and also requires more tool actions.

GPT is the exception, with nearly identical resource use in the two groups. Longer exploration therefore frequently reflects failure to converge rather than additional solved tasks.

A.8.2 Performance by Task Characteristic. Table 12 gives the sample sizes and raw accuracies underlying Figure 6. The multimodal group pools 115 tasks requiring two modalities and 19 requiring three.

A.8.3 Trace-level Root-cause Audit. Table 13 reports the complete human-confirmed subtype distribution used in Figure 7.

The audit unit is one failed Grok 4.5 task. We assign exactly one primary cause: the earliest observable divergence that conflicts with the verified solution, remains uncorrected, and determines the submitted result or prevents submission. Exploratory errors that the agent later corrects are not primary causes, and evaluator outcomes such as a column-count mismatch or missing prediction are retained only as symptoms.

Audit procedure. For each of the 136 audited failures, GPT-5.6 Sol receives the observable tool/action trace, workspace, submitted prediction, reference result, evaluation configuration, and verified solution annotation. It proposes a primary category and subtype, cites the trace event at which the persistent divergence first appears, and gives a counterfactual correction. A human researcher checks every cited event and the relevant workspace evidence, then confirms or revises the proposal. The audit does not use or claim access to hidden chain-of-thought content.

Stage boundaries. *Task intent* (Q) covers an incorrect formulation of the requested output, conditions, scope, or ordering constraint. *Discovery* (D) covers failure to locate the required evidence or selection of the wrong source. *Extraction* (E) applies when the correct artifact is accessed but its raw values are not recovered accurately; *grounding* (G) applies when those values are read but assigned the wrong field, entity, unit, or source interpretation. *Computation* (C) requires correct evidence and semantics followed by an incorrect relational or numerical operation. *Materialization* (M) is used only

Table 10: Compatibility assessment of specialized data-agent systems. “Public” refers to an official implementation released by the system’s authors.

System	Official implementation	Reason not directly included
MLE-STAR [30]	Public	Its interface targets machine-learning competition pipelines, including web search, model refinement, and predictive artifacts, rather than offline analytics over heterogeneous documents and video with complete tabular outputs.
Teable [45]	Partial	The community database platform is public, but its complete agent functions are license-gated rather than available as a pinned, independently reproducible implementation. It also provides no native video-analysis interface.
DeepAnalyze [58]	Public	The released 8B model is a text-generation agent whose documented inputs cover databases, tabular files, and text formats, but not native image or video understanding; its primary output is an analytical report rather than a complete tabular result.
TAIJI [57]	Unavailable	The paper describes an MCP-based multimodal architecture and preliminary prototype, but provides no official runnable implementation.
AOP [47]	Unavailable	No standalone implementation of the published AOP system is released; the authors’ public Unify prototype is a related but distinct system.
AgenticData [40]	Unavailable	The published multi-agent planner, optimizer, executor, and memory system has no official runnable implementation.

Table 11: Efficiency statistics for the controlled backbone comparison with DATASPACE-AGENT fixed. Token, action, and latency values are per-task averages over all 410 tasks; parentheses give the token median and latency 90th percentile. API cost per task uses each provider’s official pricing.

Backbone	Acc. (%)	Tokens (K)	Cost (USD)	Actions	Latency (s)
Grok 4.5	66.34	301.9 (124.5)	0.169	18.1	80.9 (185.7)
GPT-5.6 Sol	64.63	77.8 (50.5)	0.200	9.0	49.2 (90.8)
Kimi K3	53.41	235.2 (100.7)	0.235	19.7	260.1 (535.9)
MiMo-V2.5	39.27	237.9 (75.8)	0.011	19.2	90.9 (219.7)
Claude Sonnet 5	32.93	440.4 (96.9)	0.224	19.3	128.9 (330.2)
MiniMax M3	28.54	498.6 (178.6)	0.042	25.0	104.5 (253.4)

Table 12: Task Accuracy (%) by task characteristic with DATASPACE-AGENT fixed. Required modalities are those used by the verified solution path; workspace quartiles are formed by total workspace bytes.

Characteristic	Group	N	Grok	GPT	Kimi	MiMo	Claude	MiniMax
Language	Single-language	145	69.7	60.7	52.4	46.9	28.3	27.6
	Cross-language	265	64.5	66.8	54.0	35.1	35.5	29.1
Required modalities	Single-modal	276	68.8	65.2	58.0	43.8	37.3	32.6
	Multimodal	134	61.2	63.4	44.0	29.9	23.9	20.1
Document evidence	Absent	275	69.5	64.7	53.5	44.7	37.1	33.5
	Required	135	60.0	64.4	53.3	28.1	24.4	18.5
Video evidence	Absent	313	66.1	62.0	52.1	41.5	33.5	31.3
	Required	97	67.0	73.2	57.7	32.0	30.9	19.6
Workspace size	Q1 (smallest)	103	78.6	80.6	69.9	55.3	52.4	45.6
	Q2	102	60.8	54.9	50.0	29.4	26.5	18.6
	Q3	102	54.9	62.7	48.0	34.3	36.3	29.4
	Q4 (largest)	103	70.9	60.2	45.6	37.9	16.5	20.4
Join	Absent	297	69.0	67.7	57.6	42.8	38.4	31.6
	Required	113	59.3	56.6	42.5	30.1	18.6	20.4
Aggregation	Absent	264	64.0	64.0	52.3	39.8	35.6	30.7
	Required	146	70.5	65.8	55.5	38.4	28.1	24.7
Answer rows	One	178	73.6	66.3	52.2	46.1	20.2	30.9
	Multiple	232	60.8	63.4	54.3	34.1	42.7	26.7
Answer columns	One	197	65.5	58.4	45.7	38.1	15.2	21.3
	Multiple	213	67.1	70.4	60.6	40.4	49.3	35.2
Row order	Insensitive	318	63.5	60.4	49.4	37.4	25.5	25.5
	Sensitive	92	76.1	79.3	67.4	45.7	58.7	39.1

when the trace already contains the correct target schema and internal table, or values directly projectable to it. *Termination* (T) is

used only when no earlier persistent Q/D/E/G/C error explains the failure.

Table 13: Human-confirmed root-cause subtypes for 136 audited Grok 4.5 failures.

Stage	Subtype	Operational definition	Tasks	Share
Q	Q1	Target output, requested entities, or row granularity is misunderstood.	17	12.5%
	Q2	A condition, comparison, temporal scope, or target population is misunderstood.	13	9.6%
	Q3	An ordering or answer constraint is misunderstood.	1	0.7%
D	D2	The wrong artifact or source is selected as authoritative evidence.	3	2.2%
E	E1	A read, retrieval window, page, frame, or query omits required records.	5	3.7%
	E2	The representation of the correct artifact is parsed incorrectly.	3	2.2%
	E3	Document, image, or video evidence is transcribed incorrectly.	1	0.7%
G	G1	A field or schema element is assigned the wrong meaning.	5	3.7%
	G2	Entities, identifiers, join keys, or records are aligned incorrectly.	1	0.7%
	G3	A correctly read value is normalized with the wrong unit, date, language, or scale.	2	1.5%
	G4	Conflicting sources or versions are reconciled incorrectly.	4	2.9%
C	C1	Filtering, Boolean, or NULL logic is applied incorrectly.	2	1.5%
	C2	A join, set operation, or deduplication step is incorrect.	1	0.7%
	C3	Aggregation, grouping, windowing, or ranking is incorrect.	1	0.7%
	C6	A corrected intermediate result is not propagated through a multi-step computation.	1	0.7%
M	M1	A correct internal result is submitted with extra or missing columns.	60	44.1%
	M2	A correct internal result is submitted with extra or missing rows.	1	0.7%
	M3	Type, numeric precision, date precision, or value formatting is altered during output.	9	6.6%
	M4	The correct rows are serialized in an incorrect required order.	1	0.7%
T	T1	Unproductive iteration exhausts the action budget without an earlier persistent analytical error.	4	2.9%
	T3	The agent terminates or fails to submit after obtaining the answer.	1	0.7%
Total			136	100.0%

B Competition Deployment and Artifact Availability

Competition deployment. DATASPACE served as the official evaluation benchmark for the KDD Cup 2026 Data Agents for Complex Data Analysis competition [10]. The competition evaluated containerized agent systems through hidden A- and B-board evaluation under challenge-specific runtime, submission, and leaderboard rules. The release described in this paper instead uses the finalized semantics-aware evaluation protocol in Section 4.5.

Artifact availability. All 410 task inputs are publicly available at <https://huggingface.co/datasets/HKUSTDial/DataSpace>. The release includes reference answers and evaluation configurations for 60 representative tasks, enabling local end-to-end evaluation; the remaining 350 references are withheld for official full-benchmark evaluation. The official evaluator, baseline implementations, experiment configurations, and documentation are available at <https://github.com/HKUSTDial/DataSpace>. Both repositories are released under the MIT License.